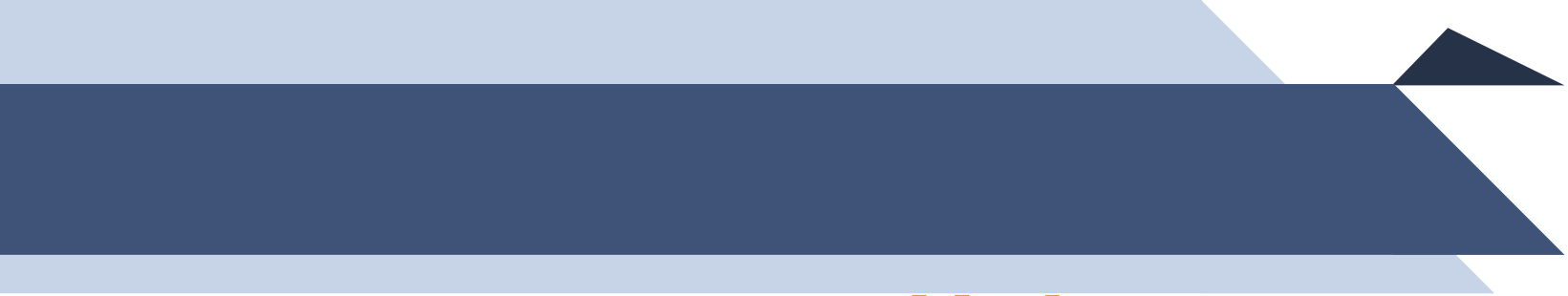


ADVANCED DATABASE SYATEMS

(22MCA102)

BY

Anantha Murthy
Assistant Professor
Dept. of MCA
NMAM.I.T, Nitte



Unit 5

Introduction to NoSQL

Introduction to NoSQL (Not Only SQL)

- NoSQL is basically a database used to manage huge sets of unstructured data, where in the data is not stored in tabular relations like relational databases.
- Most of the currently existing Relational Databases have failed in solving some of the complex modern problems like :
 - ✓ *Continuously changing nature of data - structured, semi-structured, unstructured and polymorphic data.*
 - ✓ *Applications now serve millions of users in different geo-locations, in different timezones and have to be up and running all the time, with data integrity maintained*
 - ✓ *Applications are becoming more distributed with many moving towards cloud computing.*
- It is designed for distributed data stores where very large scale of data storing needs (for example Google or Facebook which collects terabits of data every day for their users).



Features of NOSQL

- **1. NOSQL Databases are non-relational:**

They are either key-value pairs or document oriented or column-oriented or graph-based.

- **2. Distributed:**

Data is distributed across several nodes in a cluster

- **3. No support for ACID properties:**

Adherence to CAP theorem

- **4. No fixed table schema and No joins**

Support for flexibility to the schema





CAP Theorem (For Reference)

- It is also called as **Brewer's Theorem** or **two out of three theorem**.
 - *“ It states that in a distributed computing environment, it is impossible to provide the following guarantees. At the best you can have two of the following three – one must be sacrificed.”*
 1. Consistency
 2. Availability
 3. Partition Tolerance
 - **Example: Updating a status in a Facebook.** We do not want any type of consistency and also we do not want that our whole servers are available because it may be happens that it is available to some people and later on it may be show to other people also.
- 

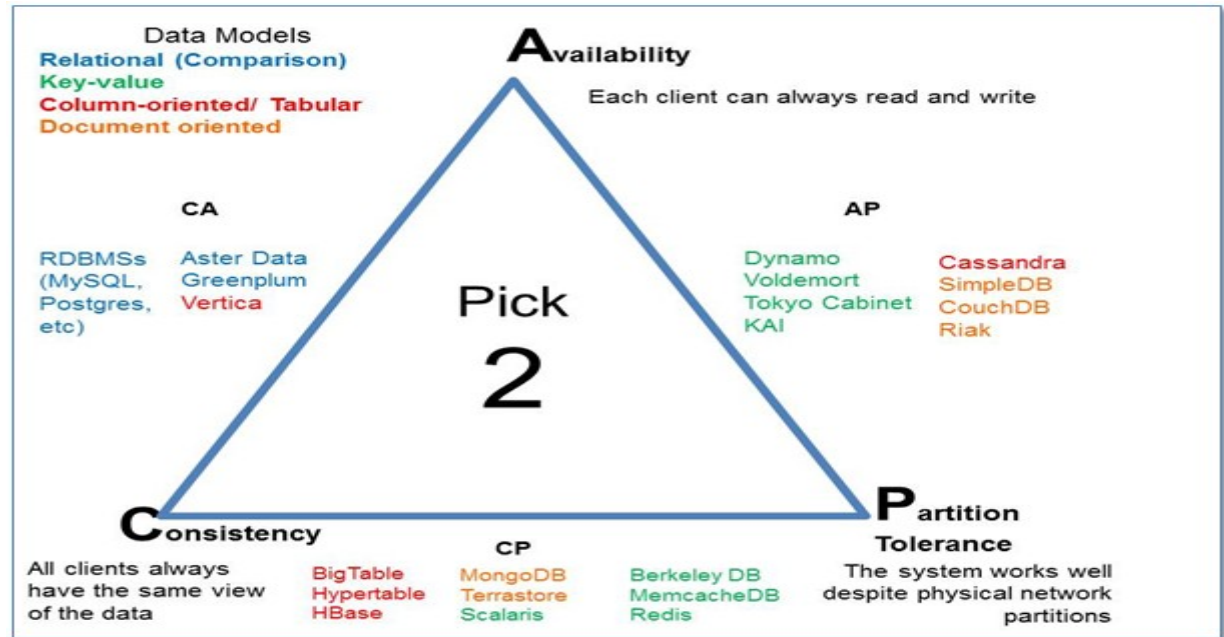
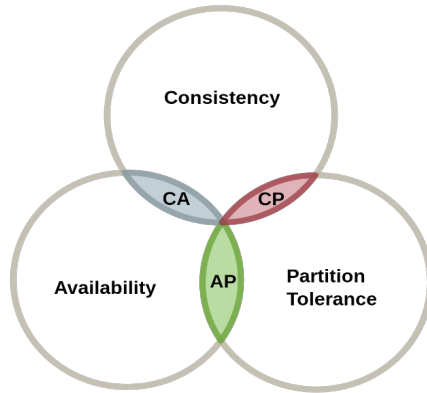


CAP Theorem (For Reference)

- **Consistency** – This means that the data in the database remains consistent after the execution of an operation. For example after an update operation all clients see the same data.
 - **Availability** – This means that the system is always on (service guarantee availability), no downtime.
 - **Partition Tolerance** – This means that the system continues to function even the communication among the servers is unreliable, i.e. the servers may be partitioned into multiple groups that cannot communicate with one another.
 - Theoretically it is impossible to fulfill all 3 requirements. CAP provides the basic requirements for a distributed system to follow 2 of the 3 requirements. Therefore all the current NoSQL database follow the different combinations of the C, A, P from the CAP theorem.
- 

CAP Theorem (For Reference)

- Here is the brief description of three combinations CA, CP, AP :
- **CA** – Single site cluster, therefore all nodes are always in contact. When a partition occurs, the system blocks.
- **CP** – Some data may not be accessible, but the rest is still consistent/accurate.
- **AP** – System is still available under partitioning, but some of the data returned may be inaccurate.



BASE PROPERTIES (For Reference)

- BASE stands for **Basically Available, Soft state and Eventually consistent**.
- Based upon CAP theorem, **NoSQL has BASE properties**.
- Based upon that BASE properties it works in a manner that it may be eventually consistent having soft state when it is basically available.

Basically Available:

- When data is stored at one site, If that site is down, our data is totally unavailable but if our data is distributed to many sites, means some part of our data is at one site and some part of data is at other site.
- If one or two sites down our whole data is not unavailable.
- It may be that some part of data becomes unavailable for some short time.
- Our data is now partially available. It leads to partially failure without any totally failure.

BASE PROPERTIES (For Reference)

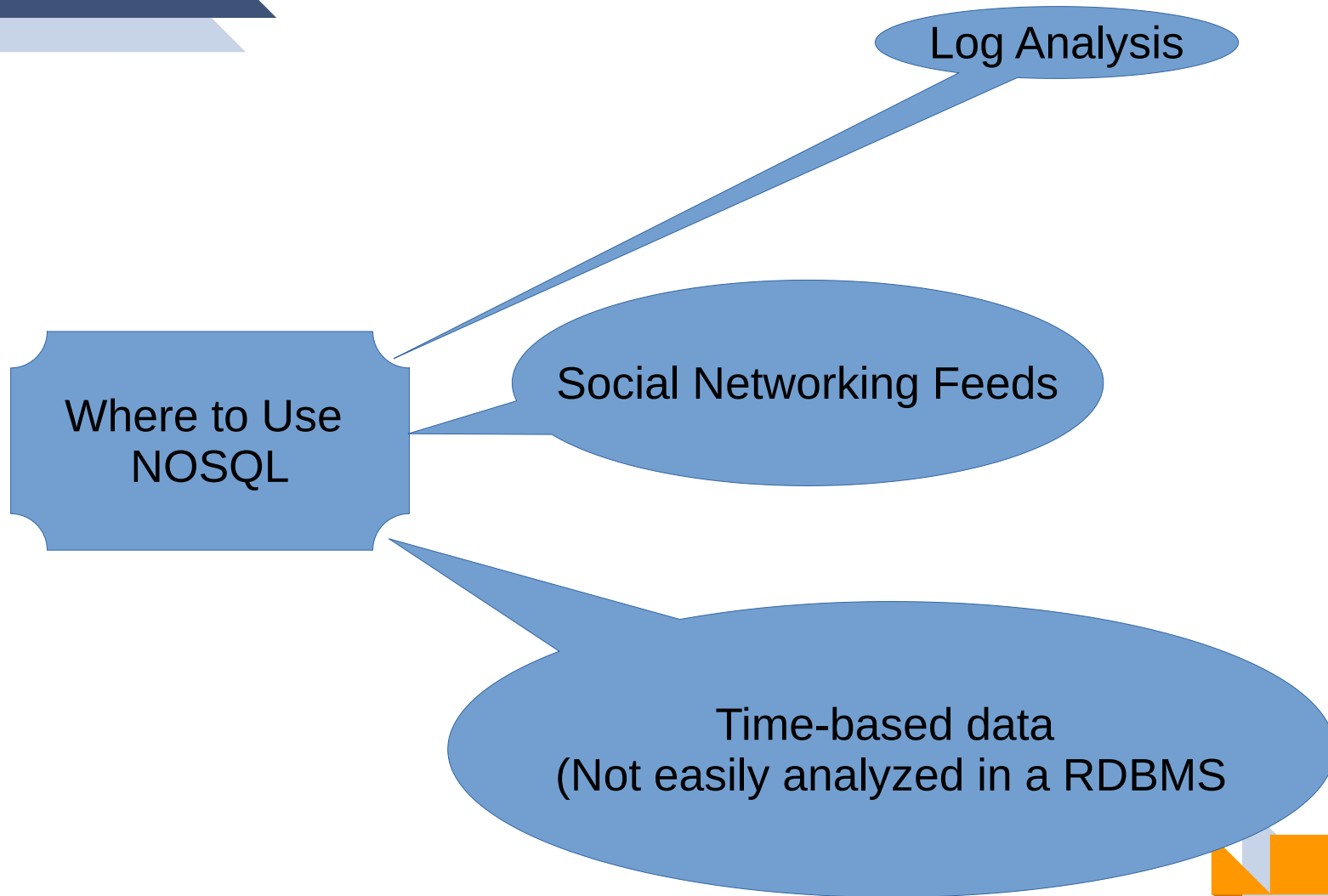
Soft State:

- Consistency is hard requirement of ACID properties but BASE allows that our data may be inconsistent.
- It allows dynamic designing to application developers.
- In other words, soft state indicates that the state of the system may change over time, even without input.

Eventually Consistent:

- NoSQL systems assume that they provided consistency in the future, not immediately at the end of operation.
- In contrast to ACID systems that enforce consistency after transaction commit, NoSQL guarantees consistency only at some undefined future time.
- So eventually consistency means it is consistent after some time even at that time it is not guarantee of the consistency.

Where NOSQL is Used



Why NOSQL ?/ Advantages of NOSQL

1. Can easily scale up and down

- **Cluster scale:** Allows distribution of database across 100+ nodes often in multiple data centers.
- **Performance scale:** Sustains over 1000,000+ database reads and writes per second
- **Data scale:** Supports housing of 1 billion+ documents in the database

2. Doesn't require a predefined schema

3. Cheap, easy to implement

- Deploying NoSQL properly allows for all of the benefits of scale, high availability, fault tolerance etc while also lowering operational costs.

Why NOSQL ?/ Advantages of NOSQL

4. Relaxes the data consistency requirement

- NoSQL have adherence to CAP theorem
- Compromise on consistency in favor of availability and partition tolerance.
- They do go for Eventual consistency

5. Data can be replicated to multiple nodes and can be partitioned

(a)Sharding:

NoSQL supports auto-sharding (automatically spread data across an arbitrary number of servers.

- No application disruption when a server goes down

(b)Replication

- Multiple copies of the data are stored across the cluster and even across data centers.
- Promises high availability and fault tolerance



Types of NoSQL Databases

- There are four general types (most common categories) of NoSQL databases.
 - Each of these categories has its own specific attributes and limitations.
-
- Key-value stores
 - Column-oriented
 - Graph
 - Document oriented



Types of NoSQL Databases

- Designed to handle huge amounts of data.
 - Key value stores allow developer to store schema-less data.
 - In the key-value storage, database stores data as hash table where each key is unique and the value can be string, JSON, BLOB (Binary Large OBJec) etc.
 - A key may be strings, hashes, lists, sets, sorted sets and values are stored against these keys. **For example** a key-value pair might consist of a key like "Name" that is associated with a value like "Ram".
 - Key-Value stores can be used as collections, dictionaries, associative arrays etc.
 - Key-Value stores follow the 'Availability' and 'Partition' aspects of CAP theorem.
 - Key-Values stores would work well for shopping cart contents, or individual values like color schemes, a landing page URI, or a default account number.
 - **Example of Key-value store DataBase** : Redis, Dynamo, Riak, Membase. etc.
- 

Types of NoSQL Databases

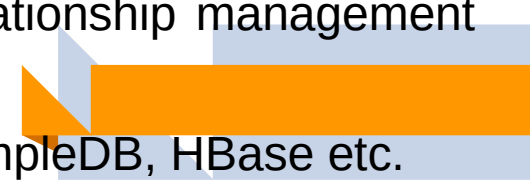
Key-value stores





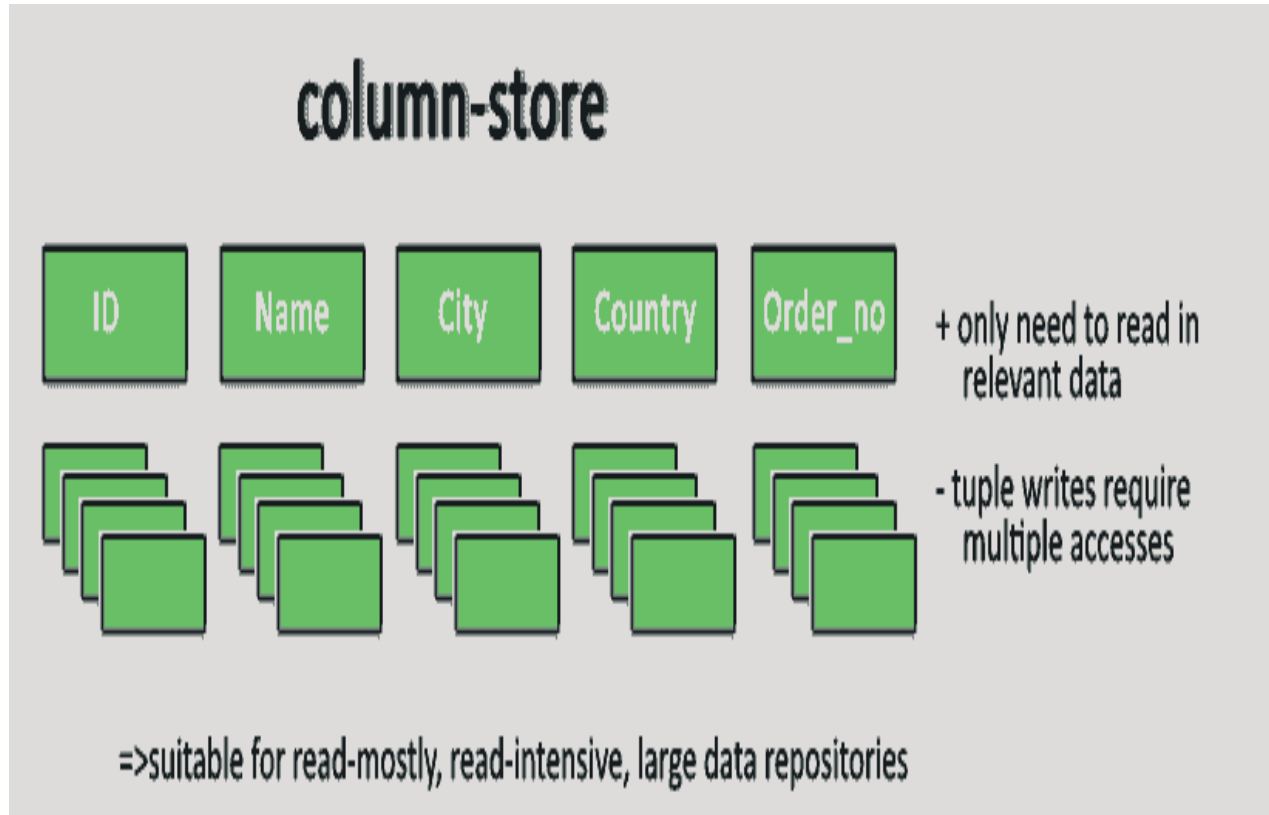
Types of NoSQL Databases

Column-oriented databases

- Column-oriented databases primarily work on columns and every column is treated individually.
 - Values of a single column are stored contiguously.
 - Column stores data in column specific files.
 - In Column stores, query processors work on columns too.
 - All data within each column datafile have the same type which makes it ideal for compression.
 - Column stores can improve the performance of queries as it can access specific column data.
 - High performance on aggregation queries (e.g. COUNT, SUM, AVG, MIN, MAX).
 - Works on data warehouses and business intelligence, customer relationship management (CRM), Library card catalogs etc.
 - **Example of Column-oriented databases** : BigTable, Cassandra, SimpleDB, HBase etc.
- 

Types of NoSQL Databases

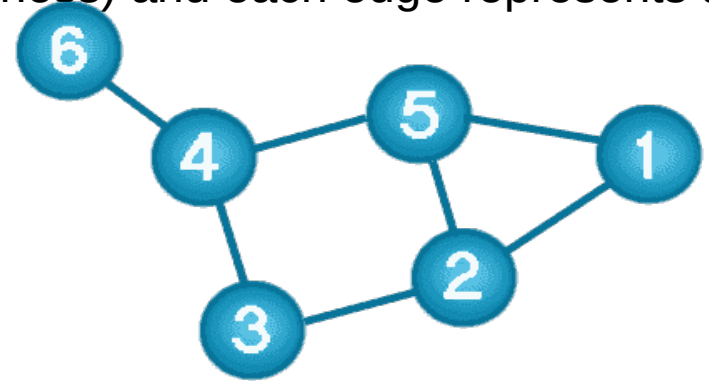
Column-oriented databases



Graph databases

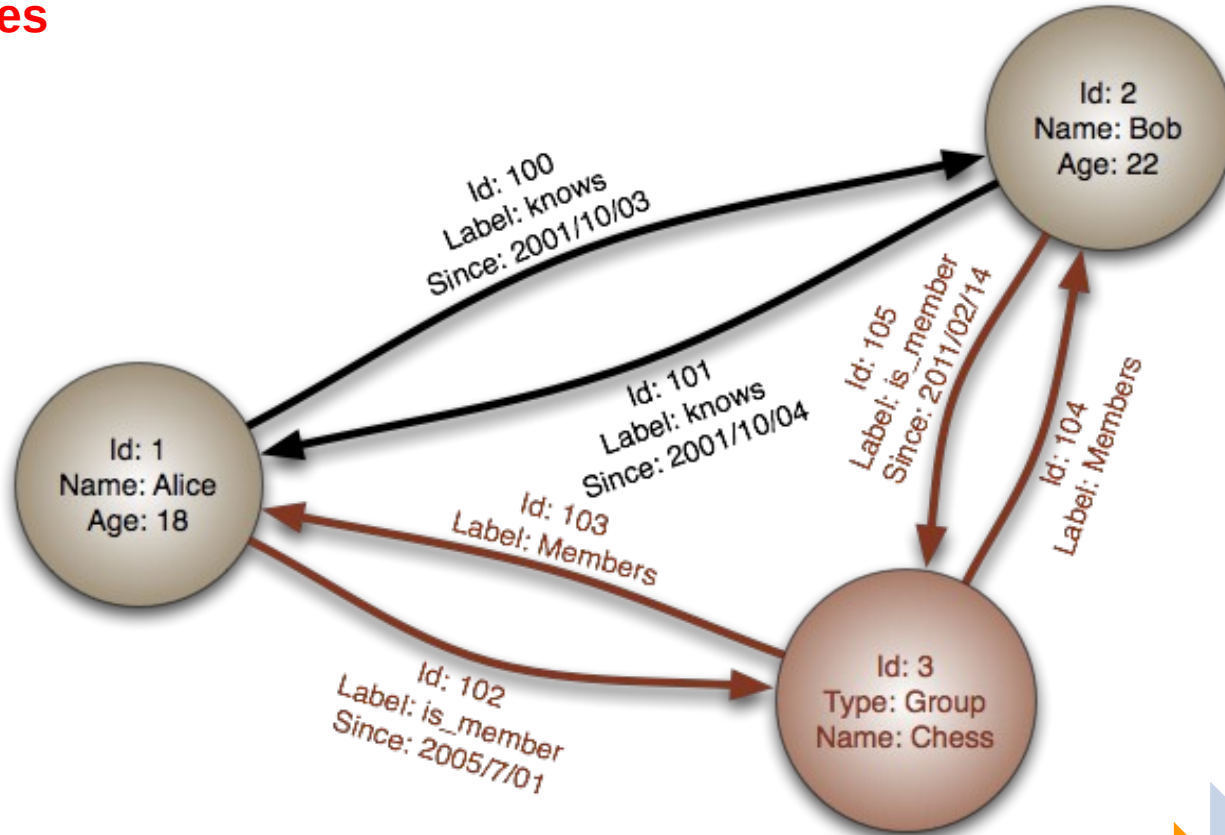
Types of NoSQL Databases

- A graph data structure consists of a finite (and possibly mutable) set of ordered pairs, called edges or arcs, of certain entities called nodes or vertices.
- Each node represents an entity (such as a student or business) and each edge represents a connection or relationship between two nodes.
- Every node and edge are defined by a unique identifier.
- Each node knows its adjacent nodes.
- As the number of nodes increases, the cost of a local step (or hop) remains the same.
- Index for lookups.



Types of NoSQL Databases

Graph databases





Types of NoSQL Databases

Graph databases

Here is a comparison between the classic relational model and the graph model :

Relational model	Graph model
Tables	Vertices and Edges set
Rows	Vertices
Columns	Key/value pairs
Joins	Edges

Example of Graph databases : OrientDB, Neo4J, Titan, InfiniteGraph. etc.





Types of NoSQL Databases

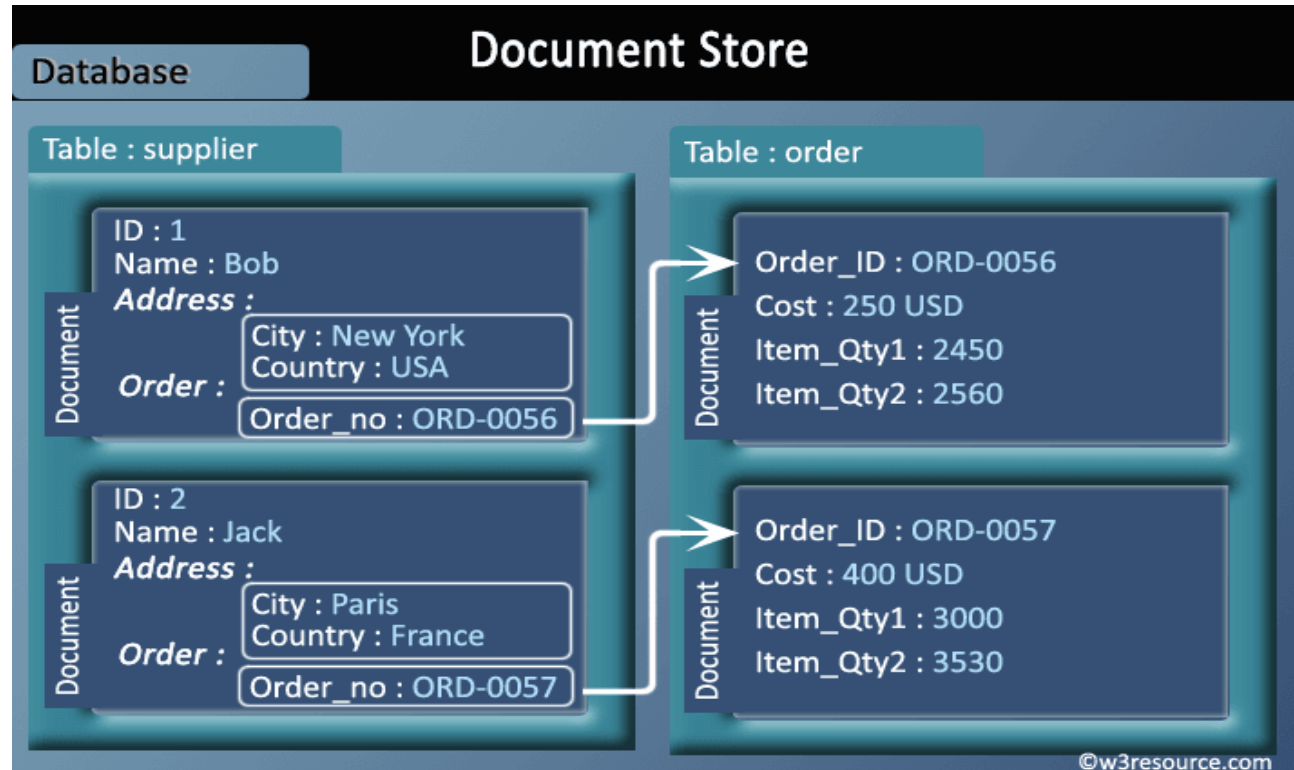
Document Oriented databases

- A collection of documents
- Data in this model is stored inside documents.
- A document is a key value collection where the key allows access to its value.
- Documents are not typically forced to have a schema and therefore are flexible and easy to change.
- Documents are stored into collections in order to group different kinds of data.
- Documents can contain many different key-value pairs, or key-array pairs, or even nested documents.



Types of NoSQL Databases

Document Oriented databases



Example of Document Oriented databases : MongoDB, CouchDB etc.



Use of NoSQL in Industry

Key-Value Pairs

- Shopping Karts
- Web User data analysis
- (Amazon. LinkdIN)

Column-Oriented

- Analyze Huge web user actions
- Sensor feeds (Facebook, Twitter)
- Ebay, Netflix

Document based

- Real-time analytics,
- Logging,
- Document archive Management

Graph-based

- Network modelling,
 - Recommendation,
 - Walmart – upsell, Cross-sell
- 

SQL vs NoSQL

SQL	NoSQL
Gigabytes to Terabytes	Petabytes(1kTB) to Exabytes(1kPB) to Zetabytes(1kEB)
Centralized	Distributed
Structured	Semi structured and Unstructured
Structured Query Language	No declarative query language
Stable Data Model	Schema less
Complex Relationships	Less complex relationships
ACID Property	Eventual Consistency
Transaction is priority	High Availability, High Scalability
Joins Tables	Embedded structures

SQL vs NoSQL

Table Based Databases	Document-based or Graph-based or Column store or key-value pairs databases
Vertically Scalable (by increasing system resources)	Horizontally scalable(by creating cluster of commodity machines)
Not preferred for large datasets	Largely preferred for large datasets
Not best fit for heirarchical data	Best fit for heirarchical data
Excellent support from vendors	Relies heavily on community service
Examples: Oracle, DB2, MySQL, MS SQL, PostgreySQL, etc	Examples: MongoDB, Hbase, Cassandra, Redis, Neo4j, CouchDB, Riak, etc



NewSQL

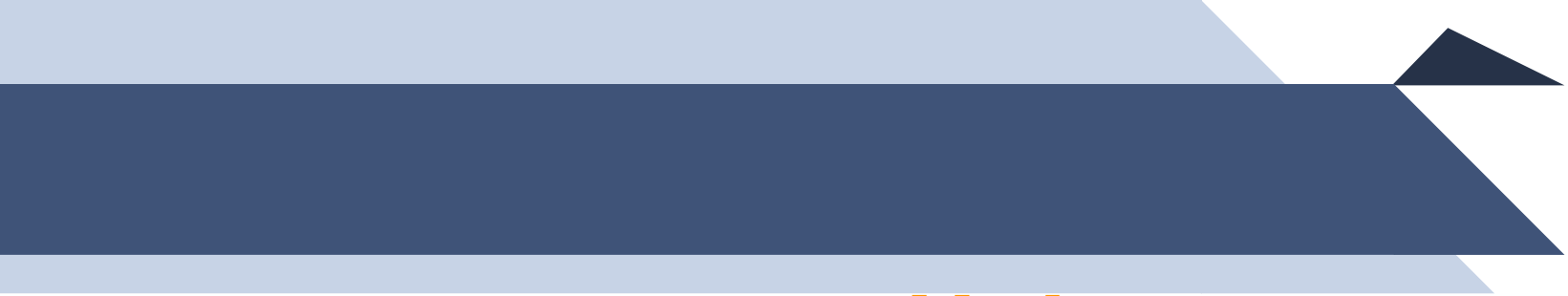
- It is a database that has the support for same scalable performance of NoSQL systems for OLTP while still maintaining the ACID guarantees of a traditional database
- It supports relational data model and uses SQL as their primary interface.

Characteristics:

- 1. SQL interface for application interaction
 - 2. ACID support for transactions
 - 3. Scale out, shared nothing architecture
 - 4. Non-locking concurrency control mechanism so that real time reads will not conflict with writes
 - 5. An architecture that provides higher per node performance with regard to traditional RDBMs solution
- 

Comparative Study of SQL, NoSQL and NewSQL

	SQL	NoSQL	NewSQL
Adherence to ACID proerties	Yes	No	Yes
OLTP/OLAP	Yes	No	Yes
Schema rigidity	Yes	No	May be
Adherence to Data Model	Relataion Model	No	No
Data format flexibility	No	Yes	May be
Scalability	Scale up vertical scaling	Scale out Horizontal scaling	Scale out
Comunity support	Huge	Growing	Slowly growing



Unit 5

Introduction to MongoDB

What is MongoDB?

MongoDB is:

- 1. Cross-platform
- 2. Open source
- 3. Non-relational
- 4. Distributed
- 5. NoSQL
- 6. Document-oriented datastore

Definition: MongoDB is an **open source, document-oriented database** designed with both scalability and developer agility in mind.

- Instead of storing your data in tables and rows as you would with a relational database, in MongoDB you store JSON-like documents with dynamic schemas (schema-free, schemaless).



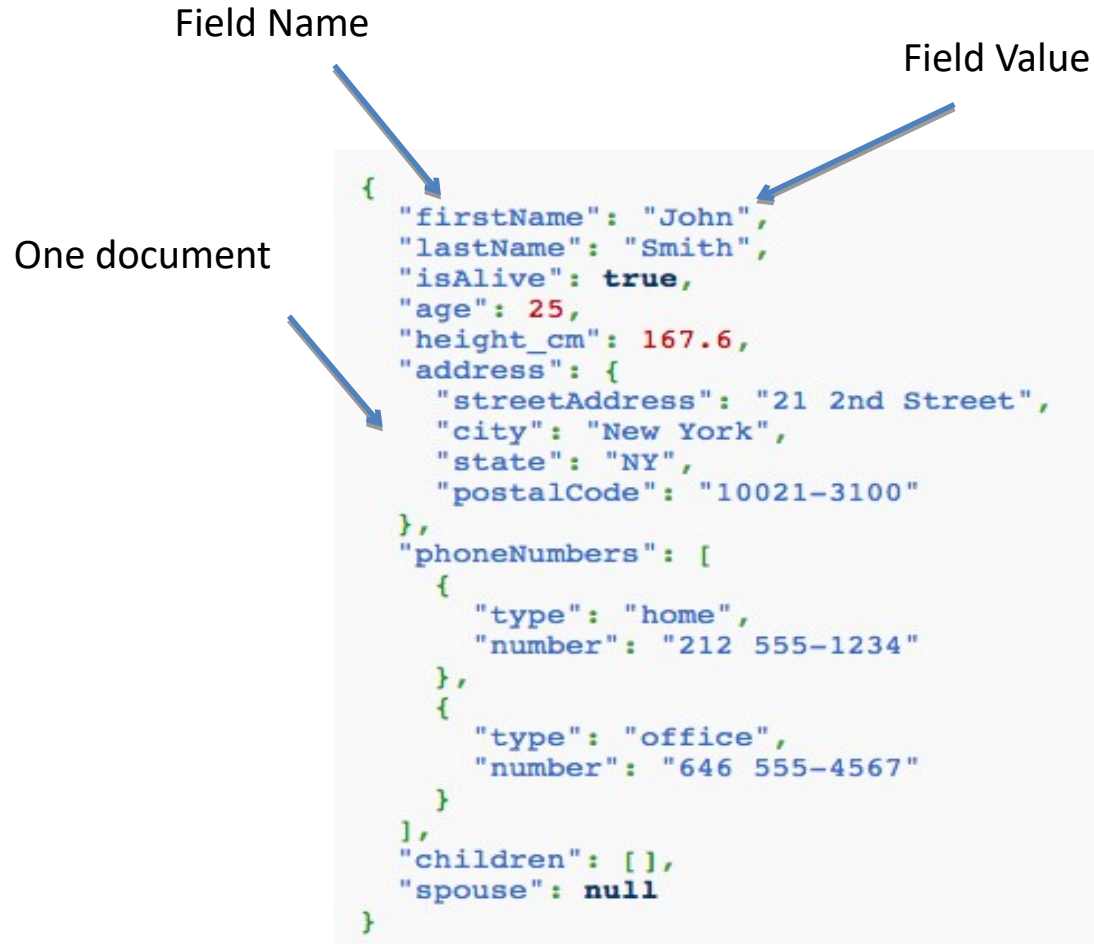
Why MongoDB?

- Few of the major challenges with traditional RDBMS are dealing with large volumes of data, rich variety of data-particularly unstructured data, and meeting up to the scale needs of enterprise data.
 - The needs is for a database that can scale out or scale horizontally to meet scale requirements, has flexibility with respect to schema, is fault tolerant, is consistent and partition tolerant , and can be easily distributed over a multitude of nodes in a cluster.
 - It is a Document Oriented Storage: Data is stored in the form of JSON style documents.
 - **JSON (JavaScript Object Notation)** is an open standard file format, and data interchange format, that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and array data types (or any other serializable value).
- 

Java Script Object Notation(JSON)

- JSON is very expressive.
- Easily store and retrieve documents in their real form.
- MongoDB does not use JSON but BSON (pronounced as Binary JSON).
- BSON is just binary JSON (a superset of JSON with some more data types, most importantly binary byte array.
 - * BSON is an open standard.
 - * In most cases it consumes less space as compared to the text-based JSON.
- It is much easier and quicker to convert BSON to a programming language's native data format.
- There are MongoDB drivers available for a number of programming languages such as C, C++, Ruby, PHP, Python, Cy, etc., and each works slightly differently.

Java Script Object Notation(JSON) Document Example





Terms used in MongoDB

- **Database** is a physical container for collections. Each database gets its own set of files on the file system. A single MongoDB server typically has multiple databases.
 - **Collection** is a group of MongoDB documents. It is the equivalent of an RDBMS table. A collection exists within a single database. Collections do not enforce a schema. Documents within a collection can have different fields. Even if the documents within a collection have same fields ,the order of the fields can be different. Typically, all documents in a collection are of similar or related purpose.
 - **A document** is a set of key-value pairs. A document is analogous to a row/record/tuple in an RDBMS table. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure, and common fields in a collection's documents may hold different types of data.
- 

Terms used in MongoDB



Figure 6.2 A collection "students" containing 3 documents.

Terms used in RDBMS and MongoDB

RDBMS	MongoDB
Database	Database
Table	Collection
Record	Document
Columns	Fields / Key Value pairs
Index	Index
Joins	Embedded documents
Primary Key	Primary key (_id is a identifier)

Creating and generating a Unique key

- Each document should have a unique identifier.
- It is similar to the primary key in relational databases.
- In MongoDB, `_id` field as the primary key for the collection so that each document can be uniquely identified in the collection.
- The `_id` field contains a unique ObjectID value.
- By default, when inserting documents in the collection, if you don't add a field name with the `_id` in the field name, then MongoDB will automatically add an ObjectID field .

Creating and generating a Unique key

```
db.Employee.find().forEach(printjson);  
  
{"_id" : ObjectId("563479cc8a8a4246bd27d784"),  
  "Employeeid" : 1,  
  "EmployeeName" : "Smith"  
  
{"_id" : ObjectId("563479d48a8a4246bd27d785"),  
  "Employeeid" : 2,  
  "EmployeeName" : "Mohan"  
  
{"_id" : ObjectId("563479df8a8a4246bd27d786"),  
  "Employeeid" : 3,  
  "EmployeeName" : "Joe"
```

Every row has a unique object id

When you query the documents in a collection, you can see the ObjectId for each document in the collection.

ObjectID

A BSON ObjectID is a 12-byte value consisting of:

- ❑ a 4-byte timestamp value, representing the ObjectID's creation, measured in seconds since the Unix epoch
- ❑ a 3-byte machine ID,
- ❑ a 2-byte process ID,
- ❑ and a 3-byte counter.

Each JSON document should have a unique identifier. It is the `_id` key.

0	1	2	3	4	5	6	7	8	9	10	11
Timestamp				Machine ID			Process ID		Counter		



ObjectID

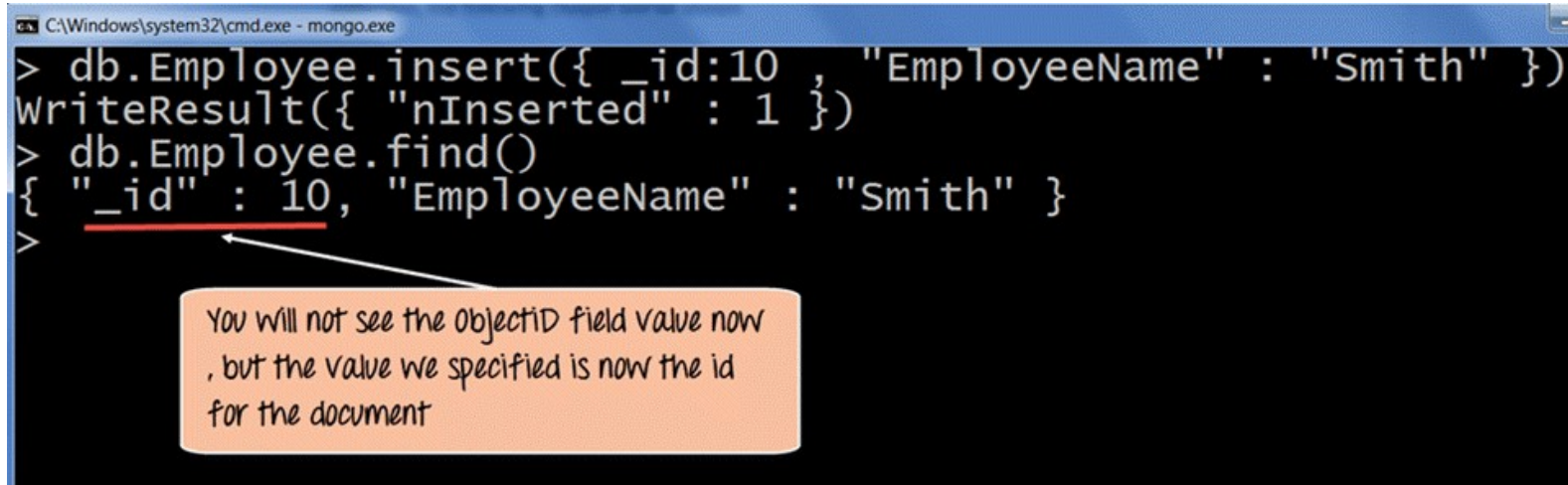
- All documents in MongoDB must have a **populated _id field**.
- If you want to ensure that MongoDB does not create the _id field when the collection is created and if you want to specify your own id as the _id of the collection, then you need to **explicitly define** this while creating the collection.
- When explicitly creating an id field, it needs to be created with **_id** in its name.
- Let's look at an example on how we can achieve this.

```
db.Employee.insert({_id:10, "EmployeeName" : "Smith"})
```

- We are assuming that we are creating the first document in the collection and hence in the above statement while creating the collection, we explicitly define the field _id and define a value for it.



ObjectID



```
C:\Windows\system32\cmd.exe - mongo.exe
> db.Employee.insert({ _id:10 , "EmployeeName" : "Smith" })
WriteResult({ "nInserted" : 1 })
> db.Employee.find()
{ "_id" : 10, "EmployeeName" : "Smith" }
>
```

You will not see the ObjectID field value now
, but the value we specified is now the id
for the document

- The output clearly shows that the `_id` field we defined while creating the collection is now used as the primary key for the collection.

Adding Documents to a Database

- To add a document to your database, use the `db.collection.insert()` command.
- The “user” in the command refers to the collection that the document was being inserted in.
- `db.user.insert({name: "Ada Lovelace", age: 20})`

```
>>> db.user.insert({name: "Ada Lovelace", age: 20});
WriteResult({ "nInserted" : 1 })
>>> show dbs;
admin      0.000GB
config     0.000GB
local      0.000GB
myDB       0.000GB
test       0.000GB
>>>
```

- `WriteResult({ "nInserted" : 1 })` indicates that the document was added to the collection.
- Now if you run the `show dbs` command you will see your database.

Create Database

- There is no “create” command in the MongoDB shell.
- In order to create a database, you will first need to switch the context to a non-existing database using the use command:
- The syntax for creating database is as follows:

`use DATABASE_Name;`

- To create a database by the name “myDB” the syntax is: `use myDB;`

```
>>> use myDB;  
switched to db myDB
```

- MongoDB only creates the database when you first store data in that database. This data could be a collection or even a document.

```
>>> use myDB;  
switched to db myDB  
>>> show dbs;  
admin    0.000GB  
config   0.000GB  
local    0.000GB  
>>>
```

Create Database

- How did the insert command know to put the data into myDB?
- When you entered the use command, then myDB became the current database on which commands operate.
- To find out which database is the current one, enter the db command(report the name of current database): **db;**

```
>>> db;  
myDB  
>>>
```

Dropping a Database

The syntax to drop database is as follows:

```
db.dropDatabase();
```

To drop the database, “myDB”, first ensure that you are currently placed in “myDB” database and then use the `db.dropDatabase()` command to drop the database.

```
use myDB;
```

```
db.dropDatabase();
```

Confirm if the database “myDB” has been dropped.

```
> db.dropDatabase();  
{ "dropped" : "myDB", "ok" : 1 }  
>
```

If no database is selected, the default database “test” is dropped.



Database

- To create a DB
 - `use myDB;`
 - To confirm the existence of DB
 - `db;`
 - To get a list of all DBs
 - `show dbs`
 - To Drop a DB
 - `db.dropDatabase();`
 - To switch to a new DB
 - `use myDB1;`
 - To display the list of collection
 - `show collections`
 - TO display statistics
 - `db.stats();`
- 

Data Types in MongoDB

String	Must be UTF-8 valid. Most commonly used data type.
Integer	Can be 32-bit or 64-bit (depends on the server).
Boolean	To store a true/false value.
Double	To store floating point (real values).
Min/Max keys	To compare a value against the lowest or highest BSON elements.
Arrays	To store arrays or list or multiple values into one key.
Timestamp	To record when a document has been modified or added.
Null	To store a NULL value. A NULL is a missing or unknown value.
Date	To store the current date or time in Unix time format. One can create object of date and pass day, month and year to it.
Object ID	To store the document's id.
Binary data	To store binary data (images, binaries, etc.).
Code	To store javascript code into the document.
Regular expression	To store regular expression.

MongoDB Query Language - CRUD

- **Create**

`db.collection.insert( <document> )`

`db.collection.save( <document> )`

`db.collection.update( <query>, <update>, { upsert: true } )`

- **Read**

`db.collection.find( <query>, <projection> )`

`db.collection.findOne( <query>, <projection> )`

- **Update**

`db.collection.update( <query>, <update>, <options> )`

- **Delete**

`db.collection.remove( <query>, <justOne> )`

The createCollection() Method

Syntax

Basic syntax of **createCollection()** command is as follows –

```
db.createCollection(name, options)
```

In the command, **name** is name of collection to be created. **Options** is a document and is used to specify configuration of collection.

Parameter	Type	Description
Name	String	Name of the collection to be created
Options	Document	(Optional) Specify options about memory size and indexing

The createCollection() Method

Options parameter is optional, so you need to specify only the name of the collection. Following is the list of options you can use –

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a fixed size collection that automatically overwrites its oldest entries when it reaches its maximum size. If you specify true, you need to specify size parameter also.
autoIndexId	Boolean	(Optional) If true, automatically create index on _id field.s Default value is false.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. If capped is true, then you need to specify this field also.
max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.

While inserting the document, MongoDB first checks size field of capped collection, then it checks max field.

The createCollection() Method

```
> db.createCollection("person");  
{ "ok" : 1 }
```

```
> db.createCollection("food");  
{ "ok" : 1 }
```

```
> show collections;  
food  
person
```

The drop() Method

MongoDB's `db.collection.drop()` is used to drop a collection from the database.

Syntax

Basic syntax of **drop()** command is as follows –

```
db.COLLECTION_NAME.drop()
```

`drop()` method will return `true`, if the selected collection is dropped successfully, otherwise it will return `false`.

```
> db.food.drop();  
true
```



Consider a document “Students” with the following attributes:

1.StudRollNo

2.StudName

3.Grade

4.Hobbies

5.DOJ



The insert() Method

To insert data into MongoDB collection, you need to use MongoDB's **insert()** ,**update()** or **save()** method.

Syntax

The basic syntax of **insert()** command is as follows –

```
>db.COLLECTION_NAME.insert(document)
```

6.5.1 Insert Method

We now explain the syntax of insert method.

db.students.insert(	←	Collection
{		
RollNo: 101,	←	Field: value
Age: 19,	←	Field: value
ContactNo:0123456789,	←	Field: value
EmailID:Sample@abc.com	←	Field: value
}		
)		

The insert() Method

If the collection does not currently exist, insert operations will **create** the collection.

```
>>> db.Students.insert({_id:1,StudName:"Michell  
Jacintha",Grade:"VII",Hobbies:"Internet Surfing"});  
WriteResult({ "nInserted" : 1 })  
>>> show collections;  
Students  
person  
>>>
```

The find() Method

To **query** data from MongoDB collection, you need to use MongoDB's find() method.

The basic syntax of find() method is as follows –

db.COLLECTION_NAME.find()

find() method will display all the documents in a non-structured way.

For Example: Check if the document for Student "Michell Jacintha" has been successfully inserted into "Student" collection.

```
>>> db.Students.find();
{ "_id" : 1, "StudName" : "Michell Jacintha", "Grade" : "VII", "Hobbies" :
  "Internet Surfing" }
>>> |
```

The pretty() Method

To display the results in a **formatted way**, you can use pretty() method.

The basic syntax of pretty() method is as follows –

db.COLLECTION_NAME.find().pretty()

For Example: To format the result, one can add the pretty() method to the operation.

```
>>> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
>>> |
```

The pretty() Method

Insert another document into the collection

```
>>> db.Students.insert({_id:2,StudName:"Mabel  
Mathews",Grade:"VII",Hobbies:"Baseball"});  
WriteResult({ "nInserted" : 1 })  
>>> db.Students.find().pretty();  
{  
  "_id" : 1,  
  "StudName" : "Michell Jacintha",  
  "Grade" : "VII",  
  "Hobbies" : "Internet Surfing"  
}  
{  
  "_id" : 2,  
  "StudName" : "Mabel Mathews",  
  "Grade" : "VII",  
  "Hobbies" : "Baseball"  
}  
>>>
```

Finding documents based on search Criteria- find() method

RDBMS Where Clause Equivalents in MongoDB

To query the document on the basis of some condition, you can use following operations.

Operation	Syntax	Example	RDBMS Equivalent
Equality	{<key>:{<seg> <value>}}	db.mycol.find({"by":"tutorials point").pretty()	where by = 'tutorials point'
Less Than	{<key>:{<lt> <value>}}	db.mycol.find({"likes": {\$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{<lte> <value>}}	db.mycol.find({"likes": {\$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{<gt> <value>}}	db.mycol.find({"likes": {\$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{<gte> <value>}}	db.mycol.find({"likes": {\$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>:{<ne> <value>}}	db.mycol.find({"likes": {\$ne:50}}).pretty()	where likes != 50
Values in an array	{<key>:{<in> [<value1>, <value2>..... <valueN>]}}	db.mycol.find({"name":{\$in: ["Raj", "Ram", "Raghu"]}}).pretty()	Where name matches any of the value in : ["Raj", "Ram", "Raghu"]
Values not in an array	{<key>:{<nin> <value>}}	db.mycol.find({"name": {\$nin:["Ramu", "Raghav"]}}).pretty()	Where name values is not in the array : ["Ramu", "Raghav"] or, doesn't exist at all

Finding documents based on search Criteria- find() method

For Example:

Consider the Students collection

```
>>> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : ObjectId("60d18287137455e22f6d36c7"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
>>>
```

Finding documents based on search Criteria- find() method

Example:1. Find the document wherein the “StudName” has value “Aryan David”

```
db.Students.find({StudName: "Aryan David"}).pretty()
```

SQL query is : **select * from students where StudName like 'Aryan David';**

Example 2. Display only StudName from all the documents of the Student's collection. The Identifier “_id” should be suppressed and NOT displayed.

```
>>> db.Students.find({}, {StudName: 1, _id: 0})
{ "StudName" : "Michell Jacintha" }
{ "StudName" : "Mabel Mathews" }
{ "StudName" : "Aryan David" }
{ "StudName" : "Hersch Gibbs" }
{ "StudName" : "Vamsi Bapat" }
```

SQL query is : **select StudName from Students;**

Finding documents based on search Criteria- find() method

Example:3. Display only StudName and Grade from all the documents of the Student's collection. The Identifier “_id” should be suppressed and NOT displayed.

```
>>> db.Students.find({}, {StudName: 1, Grade: 1, _id: 0})
{ "StudName" : "Michell Jacintha", "Grade" : "VII" }
{ "StudName" : "Mabel Mathews", "Grade" : "VII" }
{ "Grade" : "VII", "StudName" : "Aryan David" }
{ "Grade" : "VII", "StudName" : "Hersch Gibbs" }
{ "StudName" : "Vamsi Bapat", "Grade" : "VI" }
```

SQL query is : **select StudName, Grade from students;**

Example 4. Display the StudName, Grade as well as identifier, _id from the document of the Students collection where _id column is 1.

```
>>> db.Students.find({_id: 1}, {StudName: 1, Grade: 1})
{ "_id" : 1, "StudName" : "Michell Jacintha", "Grade" : "VII" }
```

SQL query is : **select StudName, Grade from Students where id=1;**

Finding documents based on search Criteria- find() method

Example:5. Display the StudName and Grade from the document of the Students collection where _id column is 1.The _id field should NOT be displayed.

```
>>> db.Students.find({_id: 1}, {StudName: 1, Grade: 1, _id: 0})
{ "StudName" : "Michell Jacintha", "Grade" : "VII" }
```

Example 6. Find the documents where the Grade is set to “VII”.

```
>>>db.Students.find({Grade:{$eq:"VII"}}).pretty();
```

SQL query is : **select * from Students where Grade like ‘VII’;**

Example 7. Find the documents where the Grade is NOT set to “VII”.

```
>>> db.Students.find({Grade: {$ne: "VII"}}).pretty()
{
  "_id" : ObjectId("60d1abbe1ab8d7c63c4c5928"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

Finding documents based on search Criteria- find() method

Example 8: Find the documents from the Students collection where Hobbies is set to either 'Chess' or set to 'Skating'.

```
>>> db.Students.find({Hobbies: {$in: ["Chess", "Skating"]}}).pretty()  
{  
  "_id" : 3,  
  "Grade" : "VII",  
  "StudName" : "Aryan David",  
  "Hobbies" : "Chess"  
}
```

Example 9. Find the documents from the Students collection where Hobbies is set to neither 'Chess' nor is set to 'Skating'.

```
>>>db.Students.find({Hobbies:{$nin:["Chess","Skating"]}}) .pretty();
```

SQL query is : **select * from Students where Hobbies not in ('Chess', 'Skating');**

Finding documents based on search Criteria- find() method

Example 10:

```
>>> db.Students.find({Hobbies : "Graffiti", StudName: "Hersch Gibbs"}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

Example 11. Find documents from the Students collection where the StudName begins with “M”.

SQL query is :

select * from Students
where StudName like 'M%';

```
>>> db.Students.find({StudName: /^M/}).pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
db.Students.find({StudName: {$regex: "^M"}}).pretty()
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

Finding documents based on search Criteria- find() method

Example 12: Find documents from the Students collection where the StudName ends with "s".

```
>>> db.Students.find({StudName: {$regex: "s$"}}).pretty()
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

SQL query is : **select * from Students where StudName like '%s';**

Finding documents based on search Criteria- find() method

Example 13: Find documents from the Students collection where the StudName has an “e” in any position.

```
>>> db.Students.find({StudName: {$regex: "e"}}).pretty()
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

SQL query is : **select * from Students where StudName like '%e%';**

Finding documents based on search Criteria- find() method

Example 13: Find documents from the Students collection where the StudName has an “e” in any position. (OR) using regex

```
>>> db.Students.find({StudName: {$regex: "e"}}).pretty()
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

SQL query is : **select * from Students where StudName like '%e%';**

Finding documents based on search Criteria- find() method

Example 14: Find documents from the Students collection where the StudName ends in “a”. using regex

```
>>> db.Students.find({StudName: {$regex: "a$"}}).pretty()
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
```

SQL query is : **select * from Students where StudName like '%a';**

Example 15: Find documents from the Students collection where the StudName begins with “M”. using regex

```
db.Students.find({StudName: {$regex: "^M"}}).pretty()
```



The Update() Method

The update() method updates the values in the existing document.

Note:

- ❑ MongoDB's update() and save() methods are used to update document into a collection.
- ❑ The update() method **updates the values** in the existing document
- ❑ while the save() method **replaces the existing document with the document passed in save() method.**

Syntax:

db.COLLECTION_NAME.update(SELECTION_CRITERIA, UPDATED_DATA, options)



The Update() Method

The `db.collection.update()` method has the following form:

```
db.collection.update(  
  <query>,  
  <update>,  
  {  
    upsert: <boolean>,  
    multi: <boolean>,  
    writeConcern: <document>,  
    collation: <document>,  
    arrayFilters: [ <filterdocument1>, ... ],  
    hint: <document|string>, // Added in MongoDB 4.2  
    let: <document> // Added in MongoDB 5.0  
  }  
)
```

By default, MongoDB will update only a single document. To update multiple documents, you need to set a parameter 'multi' to true.

```
db.Students.update({}, {$set: {field: "value"}}, {multi: true})
```

The Update() Method

Example:

Insert document for “Aryan David” into the Students collection only if it does not already exist in the collection. However ,if it already present in the collection, then update the document with new values(Update his Hobbies from “Skating” to “Chess”)

```
>>> db.Students.update({_id:3,StudName:"Aryan David",Grade:"VII"},{$set:
{Hobbies:"Skating"}},{upsert:true});
WriteResult({ "nMatched" : 0, "nUpserted" : 1, "nModified" : 0, "_id" : 3 })
>>>
```

“upsert”(Update else Insert):

- ☐ If there is an existing document, it will attempt to update it
- ☐ If there is no existing document, then it will insert it.
- ☐ The default value for this option is false.

nUpserted:1 implies that a document with _id:3 has been inserted.

```
db.Students.update(
  {StudName: "Aryan David"},
  {$set: {Hobbies: "Chess"}},
  {upsert: true}
)
```

The Update() Method

Example:

```
>>> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Skating"
}
>>> |
```

The Update() Method

Example:

```
>db.mycol.update({'title':'MongoDB Overview'},{$set: {'title':'New MongoDB Tutorial'}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>db.mycol.find()
{ "_id" : ObjectId(5983548781331adf45ec5), "title":"New MongoDB Tutorial"}
{ "_id" : ObjectId(5983548781331adf45ec6), "title":"NoSQL Overview"}
{ "_id" : ObjectId(5983548781331adf45ec7), "title":"Tutorials Point Overview"}
>
```

```
>db.mycol.update({'title':'MongoDB Overview'},
  {$set: {'title':'New MongoDB Tutorial'}},{multi:true})
```

Adding a New Field to an Existing Document -

Update Method

Example:

To add a new field “Location” with value “Newark” to the document (_id:4) of “Students” collection.

Input: Check the document(_id:4) in the “Students” collection before proceeding.

```
>>> db.Students.find({'_id':4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
```

Perform updation:

```
>>> db.Students.update({'_id':4},{ $set:{Location: "Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>>> |
```

Adding a New Field to an Existing Document -

Update Method

Example:

Confirm that the new field “Location” with the value “Newark” is added to document (_id:4) in “Students” collection.

```
>>> db.Students.find({_id:4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti",
  "Location" : "Newark"
}
>>> |
```

```
db.Students.update({},{$set:{CollegeName:"NMAMIT"}},{upsert:false,multi:true});
```

Will add a column CollegeName for all documents with a value “NMAMIT”



updateOne()

The updateOne() method is used to update one document that matches a filter condition. If multiple documents match the filter, only the first matching document will be updated.

```
db.collection.updateOne(  
  <filter>,           // The query condition to find the document to update  
  <update>,           // The update operation to apply  
  <options>           // Optional: Additional options like upsert  
)
```

filter: The query to match the document(s) that need to be updated.

update: The update operation, usually specified with operators like \$set, \$inc, \$push, etc.

options (optional): Additional settings like upsert. If upsert: true is specified, a new document will be inserted if no document matches the filter.



updateOne()

```
db.students.updateOne(  
  { name: "John Doe" },  
  { $set: { age: 22 } }  
);
```

```
db.students.updateOne(  
  { name: "Jane Smith" },  
  { $set: { age: 20 } },  
  { upsert: true }  
);
```

updateMany()

The updateMany() method is used to update multiple documents that match the given filter. Unlike updateOne(), updateMany() will update all documents that match the filter.

```
db.collection.updateMany(  
  <filter>,           // The query condition to find the documents to update  
  <update>,           // The update operation to apply  
  <options>           // Optional: Additional options like upsert  
)
```

filter: The query to match the documents that need to be updated.

update: The update operation to apply to the matching documents.

options (optional): Additional settings like upsert. If upsert: true is specified, a new document will be inserted if no documents match the filter (though this is rarely used with updateMany).

updateMany()

To increase the `age` by 1 for all students who are older than 18:

```
javascript

db.students.updateMany(
  { age: { $gt: 18 } },    // Filter: find all students older than 18
  { $inc: { age: 1 } }    // Update operation: increment age by 1
);
```

Example 2: Update Multiple Documents with Multiple Fields

To update both the `age` and `subjects` for all students who are enrolled in "Math":

```
db.students.updateMany(
  { subjects: "Math" },    // Filter: find all students with "Math" in their subjects list
  { $set: { age: 21, grade: "A" } } // Update operation: set age to 21 and grade to "A"
);
```

Update() vs updateOne() vs updateMany()

Feature	update()	updateOne()	updateMany()
Purpose	Update one or more documents (legacy method).	Update a single document that matches the filter.	Update multiple documents that match the filter.
Documents Updated	Updates one or more (depending on conditions).	Updates only the first matching document.	Updates all matching documents.
Default Behavior	Updates the first matching document. If <code>multi</code> option is set, updates multiple documents.	Always updates one document.	Always updates all matching documents.
Upsert Support	Supported via <code>upsert: true</code> option.	Supported via <code>upsert: true</code> option.	Supported via <code>upsert: true</code> option.
Replacement	Can be used to completely replace a document (if no update operator).	Can only update parts of the document with update operators.	Can only update parts of the document with update operators.



The Save() Method

The save() method will insert a new document if the document with the specified id does not exist.

■ However, if a document with the specified id exists, it replaces the existing document with the new one.

■ The save() method replaces the existing document with the new document passed in the save() method.

Syntax

```
>db.COLLECTION_NAME.save({_id:ObjectId(),NEW_DATA})
```



The Save() Method (deprecated)

Example:

To demonstrate save() method insert a document “Vamsi Bapat” in “Students” collection. Omit providing the value for the `_id` key. Consider the following collection

```
>>> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti",
  "Location" : "Newark"
}
>>>
```

The Save() Method

Example:

```
>>> db.Students.save({_id: ObjectId(), StudName: "Vamsi Bapat"})
WriteResult({ "nInserted" : 1 })
>>>
```

```
>>> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti",
  "Location" : "Newark"
}
{
  "_id" : ObjectId("60cc758536089058413578e5"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
>>>
```

The remove() Method

MongoDB's remove() method is used to remove a document from the collection.

remove() method accepts two parameters.

- **deletion criteria** – (Optional) deletion criteria according to documents will be removed.
- **justOne** – (Optional) if set to true or 1, then remove only one document.

Syntax: `db.COLLECTION_NAME.remove(DELETION_CRITTERIA)`

Consider the mycol collection has the following data.

Example:

```
{_id : ObjectId("507f191e810c19729de860e1"), title: "MongoDB Overview"},  
{_id : ObjectId("507f191e810c19729de860e2"), title: "NoSQL Overview"},  
{_id : ObjectId("507f191e810c19729de860e3"), title: "Tutorials Point Overview"}
```

The remove() Method

Example: Following example will remove all the documents whose title is 'MongoDB Overview'.

```
>db.mycol.remove({'title':'MongoDB Overview'})
WriteResult({"nRemoved" : 1})

> db.mycol.find()
{"_id" : ObjectId("507f191e810c19729de860e2"), "title" : "NoSQL Overview" }
{"_id" : ObjectId("507f191e810c19729de860e3"), "title" : "Tutorials Point Overview" }
```

Remove Only One

If there are multiple records and you want to delete only the first record, then set `justOne` parameter in `remove()` method.

```
>db.COLLECTION_NAME.remove(DELETION_CRITERIA,1)
```

The remove() Method

Example: Remove All Documents

If you don't specify deletion criteria, then MongoDB will delete whole documents from the collection. This is equivalent of SQL's truncate command.

```
> db.mycol.remove({})
```

```
WriteResult({ "nRemoved" : 2 })
```

```
> db.mycol.find()
```

The deleteOne() Method

The deleteOne() method deletes one document that matches the filter. If multiple documents match the filter, only the first document found will be deleted.

```
db.collection.deleteOne(  
  <filter>,           // The query to find the document to delete  
  <options>           // Optional settings like `collation`  
)
```

filter: The query to select the document that should be deleted.

options: Optional settings like collation, which helps you specify locale-based string comparison.

Example 1: Delete One Document

To delete the first student named "John Doe":

```
db.students.deleteOne({ name: "John Doe" });
```

The deleteMany() Method

The deleteMany() method is used to delete multiple documents that match a filter. All documents that match the condition are deleted.

Syntax:

```
db.collection.deleteMany(  
  <filter>,           // The query to find documents to delete  
  <options>           // Optional settings like `collation`  
)
```

- **filter:** The query to select the documents that should be deleted.
- **options:** Optional settings like `collation`, which helps you specify locale-based string comparison.

The deleteMany() Method

Example 1: Delete Multiple Documents

To delete all students who are older than 18:

```
db.students.deleteMany({ age: { $gt: 18 } });
```

Method	Description	Number of Documents Deleted	Example Use Case
<code>remove()</code>	Legacy method to delete documents, deprecated in MongoDB 4.2.	Single or multiple documents (by default, all that match).	Legacy use, should be avoided.
<code>deleteOne()</code>	Deletes the first document that matches the filter.	One document	Deleting a specific document (e.g., by <code>_id</code> or name).
<code>deleteMany()</code>	Deletes all documents that match the filter.	Multiple documents	Deleting documents that meet a condition (e.g., removing all inactive users).

Removing an Existing Field from an Existing Document - Update Method

Example: Remove the field "Location" with value "Newark" in the document (_id:4) of "Students" collection.

```
>>> db.Students.find({_id:4}).pretty();
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti",
  "Location" : "Newark"
}
```

```
db.Students.update(
  {_id: 4},
  {$unset: {Location: ""}}
)
```

```
>>> db.Students.update({_id:4},{$unset:{Location:"Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

Dealing Null values

A NULL is a missing or unknown value.

- When we place NULL as value for a field, it implies that currently we do not know the value or the value is missing.
- We can always update the value of the field once we know it.

Example: To add a new field with NULL value in existing documents (`_id:3` and `_id:4`) of Students collection.

Before we execute the commands, to update documents with a null value in a column, let us first view the two documents.

```
>>> db.Students.find({$or:[{ _id:3},{_id:4}]});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
```

Dealing Null values

Update the documents with NULL values in the “Location” column.

```
>>> db.Students.update({_id:3},{ $set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>>> db.Students.update({_id:4},{ $set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
db.Students.update(
  {_id: 3},
  {$set: {Location: null}}
)
db.Students.update(
  {_id: 4},
  {$set: {Location: null}}
)
```

To search for NULL values in Location column.

```
>>> db.Students.find({Location:{$eq:null}}); db.Students.find({Location: null})
{ "_id" : 1, "StudName" : "Michell Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess", "Location" : null }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti", "Location" : null }
{ "_id" : ObjectId("60d1abbe1ab8d7c63c4c5928"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
```

Sql Query : Select * from Students where Location is Null;

Dealing Null values

To remove “Location” field having NULL values from the documents (id:3 and id:4) from the Students collections.

- Remove the Location field if it contains NULL values.

```
>>> db.Students.update({_id:3},{ $unset:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>>> db.Students.update({_id:4},{ $unset:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
mongodb

db.Students.update(
  {Location: null},
  { $unset: {Location: ""}},
  {multi: true}
)
```

Let us confirm if the changes have been made by running find method on the Students collection.

```
>>> db.Students.find();
{ "_id" : 1, "StudName" : "Michell Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
{ "_id" : ObjectId("60d1ce0135a427e1ab980a6d"), "StudName" : "Vamsi Bapat", "Grade" : "VI" }
```



Count, Limit, Sort and Skip

Example 1: Find number of documents in the Students collection.

```
>>> db.Students.count();  
5
```

Example 2: Find the number of documents in the Students collection wherein the Grade is VII.

```
>>> db.Students.count({Grade: "VII"});  
4
```



Count, Limit, Sort and Skip

Example 3: Retrieve the first 3 documents from the Students collection wherein the Grade is VII.

```
>>> db.Students.find({Grade: "VII"}).limit(3).pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
```

```
db.Students.find({Grade: "VII"}).limit(3)
```

Count, Limit, Sort and Skip

Example 4: Sort the documents from the Students collection in the ascending order of StudName.

```
>>> db.Students.find().sort({StudName:1}).pretty();
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

```
db.Students.find().sort({StudName: 1})
```

Count, Limit, Sort and Skip

Example 5: Sort the documents from the Students collection in the descending order of StudName.

```
>>> db.Students.find().sort({StudName:-1}).pretty();
```

```
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
```

```
db.Students.find().sort({StudName: -1})
```

Count, Limit, Sort and Skip

Example 6: Sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in descending order.

```
>>> db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
```

db.Students.find().sort({Grade: 1, Hobbies: -1})

Count, Limit, Sort and Skip

Example 7: Sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in ascending order.

```
>>> db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
>>>
```

Count, Limit, Sort and Skip

Example 8: Skip the first 2 documents from the Students collection

```
>>> db.Students.find().skip(2).pretty();
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
>>>
```

`db.Students.find().skip(2)`

Count, Limit, Sort and Skip

Example 9: Sort the documents from the Students collection and skip the first document from the output.

```
>>> db.Students.find().skip(1).pretty().sort({StudName:1});
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

```
db.Students.find().sort({field: 1}).skip(1)
```

Count, Limit, Sort and Skip

Example 10: Display the last 2 documents from the Students collection

```
>>> db.Students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michell Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{
  "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
>>>
```

```
>>> db.Students.find().pretty().skip(db.Students.count()-2);
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

```
db.Students.find().sort({_id: -1}).limit(2)
```

Count, Limit, Sort and Skip

Example 11: Retrieve the third, fourth, and fifth documents from the Students collection

```
>>> db.Students.find().pretty().skip(2).limit(3);
{
  "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess"
}
{
  "_id" : 4,
  "Grade" : "VII",
  "StudName" : "Hersch Gibbs",
  "Hobbies" : "Graffiti"
}
{
  "_id" : ObjectId("60d1ce0135a427e1ab980a6d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
>>>
```

`db.Students.find().skip(2).limit(3)`



Arrays

MongoDB Array is a flexible document structure; it will make it possible to have a field with array as a value in MongoDB.

This is nothing but a simple list of values, and it can take many forms in MongoDB.

We can define an array of string, integer, embedded documents, Jason and BSON data types, array in it can be defined as any form of data types.

If we have made student collection and make the field as grade and grade are divided into three parts, such as MongoDB array, Array is essential and useful in MongoDB.

Syntax:

```
{< array field >: {< operator1> : <value1>, < operator2> : <value2>, < operator3> :  
<value3>, ..... }}
```



Arrays

Example 1: To create a collection by the name “food” and then insert documents into the “food” collection, Each document should have a “fruits” array.

```
>>> db.food.insert({_id:1,fruits:["banana","apple","cherry"]});
WriteResult({ "nInserted" : 1 })
>>> db.food.insert({_id:2,fruits:["orange","butterfruit","mango"]});
WriteResult({ "nInserted" : 1 })
>>> db.food.insert({_id:3,fruits:["pineapple","strawberry","grapes"]});
WriteResult({ "nInserted" : 1 })
>>> db.food.insert({_id:4,fruits:["banana","strawberry","grapes"]});
WriteResult({ "nInserted" : 1 })
>>> db.food.insert({_id:5,fruits:["orange","grapes"]});
WriteResult({ "nInserted" : 1 })
>>> | db.food.insert({_id: 1, fruits: ["banana", "apple", "cherry"]})
```

```
>>> db.food.find();
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{ "_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
>>>
```

Arrays

Example 2: Find those documents from the “food” collection which has the “fruits” array constituted “banana”, “apple” and “cherry”.

```
>>> db.food.find({fruits:["banana","apple","cherry"]}).pretty();
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
>>>
```

Example3: Find those documents from the “food” collection which has the “fruits” array having “banana” as an element

```
>>> db.food.find({fruits:"banana"});
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
```

Arrays

Example 4: Find those documents from the “food” collection which has the “fruits” array having “grapes” in the first index position. The index position begins at 0.

```
>>> db.food.find({"fruits.1":"grapes"});  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
```

Example 5: Find those documents from the “food” collection which has the “fruits” array having “grapes” in the second index position.

```
>>> db.food.find({"fruits.2":"grapes"});  
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }  
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
```

Arrays

Example 6: Find those documents from the “food” collection which has the “fruits” array where the size of the array is two. The size implies that the array holds only 2 values.

```
>>> db.food.find({fruits:{$size:2}});  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
```

Example 7: Find those documents from the “food” collection which has the “fruits” array where the size of the array is three. The size implies that the array holds only 3 values.

```
>>> db.food.find({fruits:{$size:3}});  
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }  
{ "_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }  
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }  
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
```

Arrays

Example 8: Find those documents with (`_id:1`) from the “food” and display two elements from the array “fruits”, starting with the element at 0 th index position

```
>>> db.food.find({_id:1},{fruits:{$slice:[0,2]}});  
{ "_id" : 1, "fruits" : [ "banana", "apple" ] }
```

Example 9: Find those documents with (`_id:1`) from the “food” and display two elements from the array “fruits”, starting with the element at 1 st index position

```
>>> db.food.find({_id:1},{fruits:{$slice:[1,2]}});  
{ "_id" : 1, "fruits" : [ "apple", "cherry" ] }
```

Arrays

Example 10: Find those documents with (`_id:1`) from the “food” and display three elements from the array “fruits”, starting with the element at 2nd index position. Since we have only three elements in the fruits array for the document `_id:1`, it displays only one element, the element at 2nd index position, that is “cherry”

```
>>> db.food.find({_id:1},{fruits:{$slice:[2,3]}});  
{ "_id" : 1, "fruits" : [ "cherry" ] }  
>>> |
```

Update on the Array

Before we begin the update operations on the “fruits” array of the documents of “food” collection, let us take a look at the documents that we have in “food” collection.

```
>>> db.food.find();
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{ "_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
>>>
```

Update on the Array

Example 1: Update the document with “_id:4” and replace the element present in the 1 st index position of the “fruits” array with “apple”.

```
>>> db.food.update({_id:4},{ $set:{ "fruits.1": "apple" }});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.find({_id:4});  
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }  
>>> |
```

Update on the Array

Example 2: Update the document with “_id:1” and replace the element “apple” of the “fruits” array with “An apple”.

```
>>> db.food.update({_id:1,"fruits":"apple"},{$set:{"fruits.1":"An apple"}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.find({_id:1});  
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }  
>>> |
```

Update on the Array

Example 3: Update the document with “_id:2” and push new key value pairs in the “fruits” array.

```
>>> db.food.update({_id:2},{ $push:{price:{orange:60,butterfruit:200,mango:120}}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.find().pretty();  
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }  
{  
  "_id" : 2,  
  "fruits" : [  
    "orange",  
    "butterfruit",  
    "mango"  
  ],  
  "price" : [  
    {  
      "orange" : 60,  
      "butterfruit" : 200,  
      "mango" : 120  
    }  
  ]  
}  
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }  
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }  
>>>
```

Update on the Array

Example 4: Update the document with “_id:4” by adding an element “orange” to the list of elements in the “fruits” array.

```
>>> db.food.update({_id:4},{ $addToSet:{fruits:"orange"}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.find().pretty();  
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }  
{  
  "_id" : 2,  
  "fruits" : [  
    "orange",  
    "butterfruit",  
    "mango"  
  ],  
  "price" : [  
    {  
      "orange" : 60,  
      "butterfruit" : 200,  
      "mango" : 120  
    }  
  ]  
}  
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }  
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes", "orange" ] }  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }  
>>>
```

Update on the Array

Example 5: Update the document with “_id:4” by popping an element from the list of elements present in the “fruits” array. The element popped is the one from the end of the array.

```
>>> db.food.update({_id:4},{ $pop:{fruits:1}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.update({_id:4},{ $pop:{fruits:1}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })  
>>> db.food.find().pretty();  
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }  
{  
  "_id" : 2,  
  "fruits" : [  
    "orange",  
    "butterfruit",  
    "mango"  
  ],  
  "price" : [  
    {  
      "orange" : 60,  
      "butterfruit" : 200,  
      "mango" : 120  
    }  
  ]  
}  
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }  
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }  
>>>
```

Update on the Array

Example 6: Update the document with “_id:4” by popping an element from the list of elements present in the “fruits” array. The element popped is the one from the beginning of the array.

```
>>> db.food.update({_id:4},{ $pop:{fruits:-1}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })  
>>> db.food.find().pretty();  
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }  
{  
  "_id" : 2,  
  "fruits" : [  
    "orange",  
    "butterfruit",  
    "mango"  
  ],  
  "price" : [  
    {  
      "orange" : 60,  
      "butterfruit" : 200,  
      "mango" : 120  
    }  
  ]  
}  
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }  
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }  
>>>
```

Update on the Array

Example 7: Update the document with “_id:3” by popping two elements from the list of elements present in the “fruits” array. The elements popped are “pineapple” and “grapes”.

```
>>> db.food.update({_id:3},{ $pullAll:{fruits:["pineapple","grapes"]}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.find({_id:3}).pretty();  
{ "_id" : 3, "fruits" : [ "strawberry" ] }  
>>> |
```

Update on the Array

Example 8: Update the documents having “banana” as an element the “fruits” array and pop out the element “banana” from those documents.

```
>>> db.food.update({fruits:"banana"},{$pull:{fruits:"banana"}});  
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

```
>>> db.food.find().pretty();  
{ "_id" : 1, "fruits" : [ "An apple", "cherry" ] }  
{  
  "_id" : 2,  
  "fruits" : [  
    "orange",  
    "butterfruit",  
    "mango"  
  ],  
  "price" : [  
    {  
      "orange" : 60,  
      "butterfruit" : 200,  
      "mango" : 120  
    }  
  ]  
}  
{ "_id" : 3, "fruits" : [ "strawberry" ] }  
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }  
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
```



Aggregate Functions

Consider the collection “Customers” as given below. It has four documents.

We would like to filter out those documents where the “AccType” has a value other than “S”.

After the filter we should be left with three documents where the “Acctype”: “S”.

It is then required to group the documents on the basis of CustID and sum up the “AccBal” for each unique “CustID”.

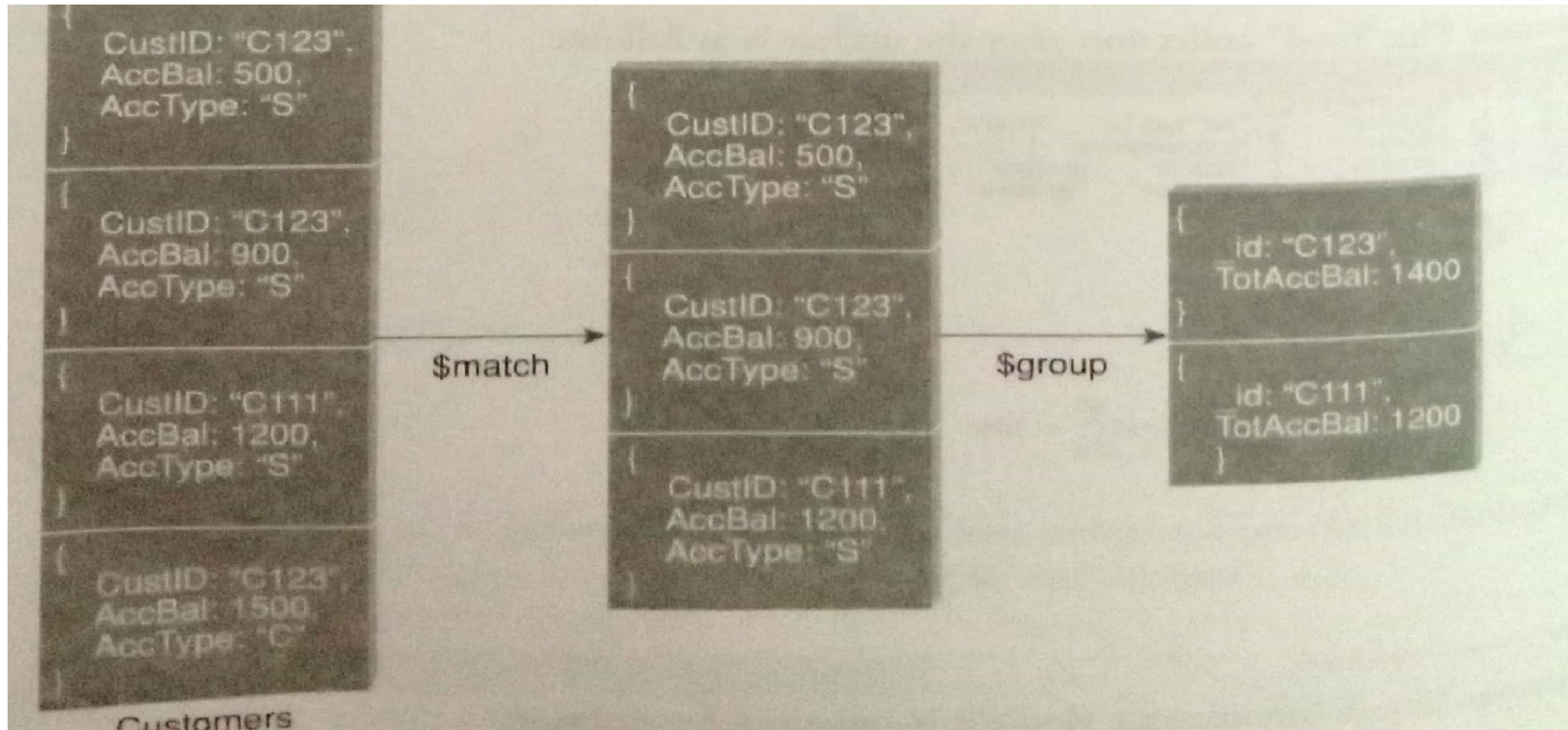
This is similar the output received with group by clause in RDBMS.

Once the groups have been formed as per the example below, there will be only two groups:

- ❑ (a) “CustID” : “C123” and
- ❑ (b) “CusID”: “C111”
- ❑ Filter and display that group where the “TotAccBal” column has a value greater than 1200.



Aggregate Functions



Aggregate Functions

Let us start off by creating collection “Customers” with the above displayed four documents:

```
>>> db.Customers.insert([{"CustID": "C123", "AccBal": 500, "AccType": "S"}, {"CustID": "C123", "AccBal": 900, "AccType": "S"}, {"CustID": "C111", "AccBal": 1200, "AccType": "S"}, {"CustID": "C123", "AccBal": 1500, "AccType": "C"}]);
BulkWriteResult({
  "writeErrors" : [ ],
  "writeConcernErrors" : [ ],
  "nInserted" : 4,
  "nUpserted" : 0,
  "nMatched" : 0,
  "nModified" : 0,
  "nRemoved" : 0,
  "upserted" : [ ]
})
```

Aggregate Functions

To confirm the presence of four documents in the “Customers” collection, use the below command:

```
//>>> db.Customers.find().pretty();
{
  "_id" : ObjectId("60d061400398473cff8baea0"),
  "CustID" : "C123",
  "AccBal" : 500,
  "AccType" : "S"
}
{
  "_id" : ObjectId("60d061400398473cff8baea1"),
  "CustID" : "C123",
  "AccBal" : 900,
  "AccType" : "S"
}
{
  "_id" : ObjectId("60d061400398473cff8baea2"),
  "CustID" : "C111",
  "AccBal" : 1200,
  "AccType" : "S"
}
{
  "_id" : ObjectId("60d061400398473cff8baea3"),
  "CustID" : "C123",
  "AccBal" : 1500,
  "AccType" : "C"
}
>>>
```

Aggregate Functions

To group on “CustID” and compute the sum of “AccBal”,use the below command:

```
>>> db.Customers.aggregate({$group:{_id: "$CustID",TotAccBal:{$sum: "$AccBal"}}});
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 2900 }
>>>
```

In order to first filter on “AccType:S” and then group it on “CustID” and then compute the sum of “AccBal”,use the below command:

```
>>> db.Customers.aggregate({$match:{AccType:"S"}},{ $group:{_id: "$CustID",TotAccBal:{$sum: "$AccBal"}}});
{ "_id" : "C111", "TotAccBal" : 1200 }
{ "_id" : "C123", "TotAccBal" : 1400 }
>>>
```

Aggregate Functions

In order to first filter on “AccType:S” and then group it on “CustID” and then compute the sum of “AccBal” and then filter those documents wherein the “TotAccBal” is greater than 1200,,use the below command:

```
>>> db.Customers.aggregate({$match:{AccType:"S"}},{ $group:{_id: "$CustID",TotAccBal:{$sum: "$AccBal"}}},{ $match:{TotAccBal:{$gt:1200}}});  
{ "_id" : "C123", "TotAccBal" : 1400 }  
>>>
```

To group on “CustID” and compute the average of the “AccBal” for each group,use the below command:

```
>>> db.Customers.aggregate({$group:{_id:"$CustID",TotAccBal:{$avg: "$AccBal"}}});  
{ "_id" : "C111", "TotAccBal" : 1200 }  
{ "_id" : "C123", "TotAccBal" : 966.6666666666666 }
```

Aggregate Functions

To group on “CustID” and determine the maximum “AccBal” for each group, use the below command:

```
>>> db.Customers.aggregate({$group:{_id:"$CustID",TotAccBal:{$max:"$AccBal"}}});  
{ "_id" : "C111", "TotAccBal" : 1200 }  
{ "_id" : "C123", "TotAccBal" : 1500 }
```

To group on “CustID” and determine the minimum “AccBal” for each group, use the below command:

```
>>> db.Customers.aggregate({$group:{_id:"$CustID",TotAccBal:{$min:"$AccBal"}}});  
{ "_id" : "C111", "TotAccBal" : 1200 }  
{ "_id" : "C123", "TotAccBal" : 500 }  
>>>
```

Find() and pretty()

In MongoDB, the `find()` and `pretty()` methods are used for querying and displaying documents in a readable format.

`find()` Method:

The `find()` method is used to retrieve documents from a collection that match a given query. By default, it returns a cursor that allows you to iterate over the results.

Syntax: `db.collection.find(query, projection)`

- **query:** (Optional) A filter or condition to match the documents. If not specified, it will return all documents in the collection.
- **projection:** (Optional) Specifies which fields to include or exclude in the returned documents. You can use `1` to include a field and `0` to exclude a field.

Example 1: Find All Documents

If you want to retrieve all documents from a collection:

```
db.students.find()
```

This will return all documents in the `students` collection.

Example 2: Find Documents with a Condition

You can specify a query to filter documents based on certain conditions. For example, to find students older than 18:

```
db.students.find({ age: { $gt: 18 } })
```

This will return all students whose `age` is greater than 18.

Example 3: Find Documents with Projection

If you only want to return specific fields (e.g., `name` and `age`) for each student, you can use a projection:

```
db.students.find({}, { name: 1, age: 1 })
```

This will return all students but only include the `name` and `age` fields.

Example 4: Find Documents with Multiple Conditions

You can combine multiple conditions in the query. For example, to find students older than 18 and whose name is "John":

```
db.students.find({ age: { $gt: 18 }, name: "John" })
```

This will return all documents where the `age` is greater than 18 and the `name` is "John".

2. `pretty()` Method:

The `pretty()` method is used to format the output of the `find()` method in a more readable way. By default, the `find()` method returns a cursor with the raw, unformatted data. Using `pretty()` will display the results in a nicely indented and human-readable format.

Syntax: `db.collection.find(query, projection).pretty()`

- **query:** (Optional) The filter or condition to match the documents.
- **projection:** (Optional) Specifies which fields to include or exclude.
- **pretty():** Used to format the results for easier reading.

Example 1: Using `pretty()` with `find()`

If you want to see the documents in a more readable format, you can chain `pretty()` after `find()`:

```
db.students.find().pretty()
```

This will return all documents in the `students` collection and format the output in a readable way with proper indentation.

Example 2: Using `pretty()` with Conditional Query

If you want to find all students older than 18 and display them in a formatted way:

```
db.students.find({ age: { $gt: 18 } }).pretty()
```

This will return all students who are older than 18 and format the output for easier reading.

Example 3: Using `pretty()` with Projection

To display only the `name` and `age` fields of students older than 18 in a readable format:

```
db.students.find({ age: { $gt: 18 } }, { name: 1, age: 1 }).pretty()
```

This will return the `name` and `age` fields for all students older than 18, formatted neatly.

Summary of `find()` and `pretty()`:

Method	Description
<code>find()</code>	Used to retrieve documents from a collection. You can specify a query to filter documents and a projection to include/exclude fields.
<code>pretty()</code>	Used to format the output of a <code>find()</code> query in a readable, indented format.

Example of Output:

When using `pretty()`, the output will be formatted like this:

```
{
  "_id": ObjectId("5f8d0d55b54764421b5e7a87"),
  "name": "John Doe",
  "age": 22,
  "subjects": ["Math", "Physics"]
}
{
  "_id": ObjectId("5f8d0d55b54764421b5e7a88"),
  "name": "Jane Smith",
```

```
"age": 21,  
"subjects": ["Chemistry", "Biology"]  
}
```

This formatted output is easier to read than the raw JSON returned by `find()` without `pretty()`.

Update(), Save() and Remove()

In MongoDB, the methods `update`, `save`, and `remove` are used to modify or remove documents in a collection. Below is an explanation of each method with its syntax and examples:

update Method:

The `update` method is used to modify documents in a collection. It can update one or more documents based on the specified conditions. As of MongoDB version 3.2, the `update` method has been replaced by the `updateOne()` and `updateMany()` methods, but `update()` still works as a legacy method.

Syntax:

```
db.collection.update(  
  query,           // Filter to select the documents to update  
  update,          // The updates to apply  
  options          // Optional (e.g., upsert)  
)
```

- **query:** Specifies the conditions that the document(s) must match to be updated.
- **update:** Specifies the changes to apply to the matched document(s). Typically, you use update operators like `$set`, `$inc`, `$push`, etc.
- **options:** Optional. You can use the `upsert` option to insert a document if no documents match the query.

Example 1: Update a Single Document (`updateOne`)

Update the `age` of the student named "John Doe" to `22`:

```
db.students.updateOne(  
  { name: "John Doe" }, // Query to match the document  
  { $set: { age: 22 } } // Update operation  
);
```

This will update only the first document that matches the query (`name: "John Doe"`) and set the `age` field to `22`.

Example 2: Update Multiple Documents (`updateMany`)

Increase the `age` by 1 for all students who are older than 18:

```
db.students.updateMany(  
  { age: { $gt: 18 } }, // Query to match all students older than 18  
  { $inc: { age: 1 } }  // Update operation: increment age by 1  
);
```

This will increment the `age` field by 1 for all documents where `age` is greater than 18.

Example 3: Upsert (Insert if Not Found)

If no student named "Jane Smith" exists, create a new document:

```
db.students.updateOne(  
  { name: "Jane Smith" }, // Query to match the document  
  { $set: { age: 20 } },  // Update operation  
  { upsert: true }        // If no match, insert a new document  
);
```

This query will insert a new student with `name: "Jane Smith"` and `age: 20` if no document with that name is found.

2. `save` Method:

The `save` method is a legacy function used for both updating an existing document or inserting a new one. If the document has an `\_id` field, MongoDB will attempt to update the document with that `\_id`. If no document is found with that `\_id`, a new document will be inserted.

Syntax: `db.collection.save(document)`

- **document:** The document to be saved. If the document has an `\_id`, MongoDB will update the existing document with that `\_id`. If no matching document is found, MongoDB will insert the document as a new document.

Example 1: Save an Existing Document

Let's say we have an existing student with `\_id: 1` and we want to update the `age`:

```
db.students.save({  
  _id: 1,          // Existing document's _id  
  name: "John Doe",  
  age: 22          // Updated age  
});
```

This will update the document with `\_id: 1` and set the `age` field to `22`.

Example 2: Save a New Document

If no document exists with the `\_id: 2`, a new document will be inserted:

```
db.students.save({  
  _id: 2,          // New document _id  
  name: "Jane Smith",  
  age: 20          // New document data  
});
```

This will insert a new student document with `\_id: 2` if no document with that `\_id` exists.

Note: The `save()` method is deprecated in MongoDB, and it is recommended to use `updateOne()`, `insertOne()`, or `replaceOne()` for better control over the operations.

3. `remove` Method:

The `remove` method is used to delete documents from a collection. As of MongoDB 3.2, the `remove` method is deprecated, and `deleteOne()` or `deleteMany()` should be used instead.

Syntax: `db.collection.remove(query, options)`

- **query:** Specifies the conditions that the documents must match to be deleted.
- **options:** Optional. You can set `justOne` to `true` to remove only one matching document (defaults to `false`, meaning all matching documents will be deleted).

Example 1: Remove a Single Document (`removeOne`)

To delete a student named "John Doe":

```
db.students.remove({ name: "John Doe" })
```

This will remove all documents with the name "John Doe" from the `students` collection.

Example 2: Remove Multiple Documents (`removeMany`)

To remove all students who are younger than 18:

```
db.students.remove({ age: { $lt: 18 } })
```

This will delete all documents where `age` is less than `18`.

Example 3: Remove with `justOne` Option

To remove only the first student found with the name "Jane Smith":

```
db.students.remove({ name: "Jane Smith" }, { justOne: true })
```

This will delete just the first matching document, even if there are multiple students named "Jane Smith."

Note: `remove` is deprecated. It is recommended to use `deleteOne()` and `deleteMany()` for better control.

4. `deleteOne` and `deleteMany` Methods (Recommended)

Instead of using `remove`, MongoDB recommends using `deleteOne()` and `deleteMany()`.

`deleteOne`: Deletes a single document that matches the query.

`db.students.deleteOne({ name: "John Doe" })`

`deleteMany`: Deletes all documents that match the query.

`db.students.deleteMany({ age: { $lt: 18 } })`

Method	Description	Syntax	Example
<code>update()</code>	Updates documents based on a query. Can update multiple.	<code>db.collection.update(query, update, options)</code>	<code>db.students.update({ name: "John" }, { \$set: { age: 20 } })</code>
<code>save()</code>	Saves a document (deprecated). Inserts or updates based on <code>_id</code> .	<code>db.collection.save(document)</code>	<code>db.students.save({ _id: 1, name: "John" })</code>
<code>remove()</code>	Deletes documents based on a query (deprecated).	<code>db.collection.remove(query)</code>	<code>db.students.remove({ age: { \$lt: 18 } })</code>
<code>deleteOne()</code>	Deletes one matching document.	<code>db.collection.deleteOne(query)</code>	<code>db.students.deleteOne({ name: "John Doe" })</code>
<code>deleteMany()</code>	Deletes all matching documents.	<code>db.collection.deleteMany(query)</code> 	<code>db.students.deleteMany({ age: { \$lt: 18 } })</code>

Dealing with NULL values

In MongoDB, handling `null` values is important, especially when querying, updating, or filtering data. Null values can represent missing data, explicitly undefined values, or fields that have not been set. MongoDB provides several ways to work with `null` values in your documents.

1. Inserting `null` Values:

You can explicitly set a field to `null` when inserting or updating a document.

Syntax: `db.collection.insertOne({ field: null });`

Example:

`db.students.insertOne({ name: "John Doe", age: null });`

In this example, the `age` field is explicitly set to `null`.

2. Querying for `null` Values:

To find documents where a field is set to `null`, you can use the query operator `field: null`.

Syntax: `db.collection.find({ field: null })`

This will return documents where the specified field is either `null` or does not exist.

Example: Find students with an age of `null`:

`db.students.find({ age: null });`

Note:

- **Field is `null`:** If the field explicitly has the value `null`, it will match.
- **Field is missing:** If the field doesn't exist in a document, it will also match, as MongoDB considers the missing field as having an implicit `null` value.

3. Querying for Non-`null` Values:

To find documents where a field is **not** `null` or the field exists, you can use the `\$ne` (not equal) operator.

Syntax: `db.collection.find({ field: { $ne: null } })`

This will return documents where the `field` is **not** `null` or the field is present.

Example: Find all students where `age` is not `null`:

`db.students.find({ age: { $ne: null } });`

This query returns documents where the `age` field is not equal to `null`.

4. Handling Fields That May Be Missing or Null:

If you want to find documents where a field either doesn't exist or is explicitly set to `null`, you can use the `\$exists` operator in combination with `null`.

Example: Find students where the `address` field does not exist or is set to `null`:

`db.students.find({ address: { $exists: false } })`

This will return documents where the `address` field does not exist.

Alternatively, if you want to find documents where the field explicitly equals `null`, you can do:

`db.students.find({ address: null })`

This query will match documents where `address` is either `null` or doesn't exist.

5. Updating Fields with `null` Values:

You can set a field to `null` during an update using the `\$set` operator.

Syntax:

```
db.collection.updateOne(  
  { query },  
  { $set: { field: null } }  
)
```

Example: To set the `age` of a specific student to `null`:

```
db.students.updateOne(  
  { name: "John Doe" },  
  { $set: { age: null } }  
);
```

This will set the `age` field of the student named "John Doe" to `null`.

6. Removing Documents with `null` Values:

If you want to delete documents that contain a `null` value for a specific field, you can use the `remove()` or `deleteMany()` method.

Syntax: `db.collection.deleteMany({ field: null })`

Example: To delete all students who have a `null` value for `age`:

```
db.students.deleteMany({ age: null });
```

This will remove all documents where `age` is set to `null` (or the field doesn't exist).

7. Handling `null` in Aggregation Pipelines:

In MongoDB's aggregation framework, `null` values can be handled using conditional operators such as `\$ifNull`, which checks if a value is `null` and allows you to provide a default value.

Example: Using `\$ifNull` to replace `null` values with a default value:

```
db.students.aggregate([  
  {  
    $project: {  
      name: 1,  
      age: { $ifNull: ["$age", 18] } // Replace null with 18  
    }  
  }  
]);
```

This aggregation will return the `name` and `age` fields, and if `age` is `null`, it will default to `18`.

Count, Limit, Sort and Skip functions

In MongoDB, the `count`, `limit`, `sort`, and `skip` operations are commonly used when querying data to control the results returned. These operations can be chained together to create efficient and customizable queries.

1. count:

The `count()` method is used to count the number of documents in a collection or the number of documents that match a query.

Syntax: `db.collection.count(query)`

- **`query`**: An optional filter to count only the documents that match specific conditions.

To count the number of students in the `students` collection:

`db.students.count()`

To count how many students are older than 18: **`db.students.count({ age: { $gt: 18 } })`**

2. limit:

The `limit()` method is used to limit the number of documents returned by a query.

Syntax: `db.collection.find(query).limit(n)`

- **`query`**: Optional filter to select specific documents.

- **`n`**: The number of documents to return.

Example: To return only 5 students from the `students` collection:

`db.students.find().limit(5)`

3. sort:

The `sort()` method is used to sort the query results by one or more fields in either ascending (`1`) or descending (`-1`) order.

Syntax: `db.collection.find(query).sort({ field1: 1, field2: -1 })`

- `query`: Optional filter to select specific documents.
- `field`: The field by which to sort.
- `1`: Sort in ascending order (default).
- `-1`: Sort in descending order.

Example: To return all students, sorted by age in ascending order:

`db.students.find().sort({ age: 1 })`

To return students sorted by age in descending order and then by name in ascending order:

`db.students.find().sort({ age: -1, name: 1 })`

4. skip:

The `skip()` method is used to skip over a specified number of documents before returning the remaining documents.

Syntax: `db.collection.find(query).skip(n)`

- `query`: Optional filter to select specific documents.
- `n`: The number of documents to skip.

Example: To skip the first 5 students and return the rest:

`db.students.find().skip(5)`

To combine all these operations, such as limiting the number of documents, sorting them, and skipping a number of them, you can chain them together.

Example (Chaining `limit`, `skip`, and `sort`):

To return 5 students who are older than 18, sorted by their names in ascending order, and skipping the first 2:

```
db.students.find({ age: { $gt: 18 } }).sort({ name: 1 }).skip(2).limit(5)
```

This query will:

- Filter students older than 18.
- Sort them by name in ascending order.
- Skip the first 2 results.
- Return the next 5 results.

Operation	Description	Syntax
<code>count()</code>	Counts the number of documents in a collection or matching a query.	<code>db.collection.count(query)</code>
<code>limit()</code>	Limits the number of documents returned.	<code>db.collection.find(query).limit(n)</code>
<code>sort()</code>	Sorts the query results by one or more fields.	<code>db.collection.find(query).sort({ field: order })</code>
<code>skip()</code>	Skips over the specified number of documents in the result set.	<code>db.collection.find(query).skip(n)</code>

These operations are useful when working with large datasets, allowing you to refine and control your query results.

Arrays

In MongoDB, arrays are a data type that allow you to store multiple values in a single field. They are ordered collections, which means the order in which values are stored is important. Arrays can contain any type of data, including strings, numbers, objects, or even other arrays.

Key Features of Arrays in MongoDB:

1. **Multiple Elements:** Arrays can hold more than one value.

2. **Heterogeneous Types:** Arrays can store mixed types of data (e.g., strings, integers, documents).

3. **Indexing:** MongoDB allows for indexing array fields, and these indexes can speed up searches on array elements.

Syntax: In MongoDB, arrays are represented by square brackets `[]`. You can store them within documents in a collection.

Example: Inserting an Array into a Document:

Let's say we have a collection called `students` where each student document includes a list of subjects.

```
db.students.insertOne({  
  name: "John Doe",  
  age: 21,  
  subjects: ["Math", "Physics", "Chemistry"]  
});
```

This document contains an array `subjects` with three values: `"Math"`, `"Physics"`, and `"Chemistry"`.

Querying an Array:

You can query documents that contain an array field using the values in that array.

For example, to find all students who study "Math":

```
db.students.find({ subjects: "Math"});
```

This will return all documents where the `subjects` array contains `"Math"`.

Updating an Array:

You can update array elements using MongoDB's update operators.

To add a new subject to the `subjects` array of a specific student:

```
db.students.updateOne(  
  { name: "John Doe" },  
  { $push: { subjects: "Biology" } }  
);
```

This adds `"Biology"` to the `subjects` array of the student named "John Doe".

Array Operators in MongoDB:

- **\$push**: Adds an element to the end of an array.
- **\$pop**: Removes the first or last element of an array.
- **\$pull**: Removes all occurrences of a value from an array.
- **\$addToSet**: Adds an element to an array only if it does not already exist.

Example of `\$pull`: To remove the subject "Physics" from John Doe's subjects:

```
db.students.updateOne(  
  { name: "John Doe" },  
  { $pull: { subjects: "Physics" } }  
);
```

This will remove "Physics" from the `subjects` array.

Arrays of Documents:

You can also store arrays of documents. For example:

```
db.students.insertOne({  
  name: "Jane Smith",  
  courses: [  
    { name: "Math 101", grade: "A" },  
    { name: "Physics 101", grade: "B" }  
  ]  
});
```

In this case, the `courses` field is an array of objects, where each object contains `name` and `grade`.

Querying Arrays of Documents:

You can query arrays of objects by specifying conditions on the fields of the objects. For example, to find students who have a grade of "A" in any course:

```
db.students.find({  
  "courses.grade": "A"  
});
```

Aggregate functions

In MongoDB, aggregate functions are used to perform operations on the data within a collection, allowing for data transformation and computation on large sets of data. MongoDB's aggregation framework provides several stages that can be combined to process and analyze documents, including filtering, grouping, sorting, and more.

Basic Syntax of Aggregation

The basic syntax for using the `aggregate()` method is:

```
db.collection.aggregate([  
    { stage1 },  
    { stage2 },  
    ...  
])
```

In this syntax:

- Each stage is represented as a document with a specific aggregation operator, and the stages are executed sequentially.
- Common aggregation stages include `$match`, `$group`, `$sort`, `$project`, and `$limit`.

Common Aggregate Functions and Examples

1. **\$match:** Filters documents based on specified criteria, similar to the `WHERE` clause in SQL.

Example: Find documents in the `Customers` collection where `AccType` is "S".

```
db.Customers.aggregate([  
    { $match: { AccType: "S" } }  
])
```

2. **\$group:** Groups documents by a specified field and performs aggregate calculations like `sum`, `avg`, `min`, `max`, etc.

Example: Group documents by `CustID` and calculate the total `AccBal` for each unique `CustID`.

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", TotalAccBal: { $sum: "$AccBal" } } }
])
```

3. **\$sort:** Sorts documents in ascending or descending order based on a specified field.

Example: Sort the grouped results by `TotalAccBal` in descending order.

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", TotalAccBal: { $sum: "$AccBal" } } },
  { $sort: { TotalAccBal: -1 } }
])
```

4. **\$project:** Reshapes documents, allowing the inclusion, exclusion, or computed fields in the output.

Example: Show only `CustID` and `TotalAccBal` in the output.

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", TotalAccBal: { $sum: "$AccBal" } } },
  { $project: { _id: 0, CustID: "$_id", TotalAccBal: 1 } }
])
```

5. **\$limit**: Limits the number of documents in the output.

Example: Retrieve the top 2 customers with the highest `TotalAccBal`.

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", TotalAccBal: { $sum: "$AccBal" } } },
  { $sort: { TotalAccBal: -1 } },
  { $limit: 2 }
])
```

Objective: Filter documents where `AccType` is "S", group them by `CustID`, calculate the total account balance (`AccBal`), and then filter the groups where the total balance is greater than 1200.

Query:

```
db.Customers.aggregate([
  { $match: { AccType: "S" } },
  { $group: { _id: "$CustID", TotAccBal: { $sum: "$AccBal" } } },
  { $match: { TotAccBal: { $gt: 1200 } } }
])
```

Explanation:

- **\$match**: Filters for documents where `AccType` is "S".
- **\$group**: Groups the documents by `CustID` and calculates the sum of `AccBal` as `TotAccBal`.
- Second **\$match**: Filters out groups where `TotAccBal` is not greater than 1200.

Additional Aggregate Functions in MongoDB

1. **\$avg**: Calculates the average of the specified field values.

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", AvgAccBal: { $avg: "$AccBal" } } }
])
```

2. **\$min and \$max**: Determine the minimum and maximum values for a specified field.

Example (Minimum):

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", MinAccBal: { $min: "$AccBal" } } }
])
```

Example (Maximum):

```
db.Customers.aggregate([
  { $group: { _id: "$CustID", MaxAccBal: { $max: "$AccBal" } } }
])
```

These aggregation functions in MongoDB allow you to perform powerful data analysis directly within the database, similar to SQL's `GROUP BY`, `WHERE`, and aggregate functions, but with greater flexibility for handling unstructured data.